OpenSearch

Cahier de Travaux Pratiques

David THIBAU – 2026
david.thibau@gmail.com

Pré-requis

- Bonne connexion Internet
- 16 Go RAM minimum (32 Go recommandés), espace disque important (> 20 Go libres)
- OS : Linux, macOS, Windows 10/11
- IDE : VSCode + IntelliJ + STS
- Docker, Git
- JMeter

Images docker utilisées

- opensearchproject/opensearch:3.4.0
- opensearchproject/opensearch-dashboards:3.4.0
- apache/kafka:4.2.0
- redpandadata/console:v3.5.3
- confluentinc/cp-kafka-connect:7.8.0

Table des matières

Pré-requis....1
Images docker utilisées....1
Ateliers 1 — Mise en place du cluster....3
1.1 Démarrage via docker compose....3
1.2 Santé du cluster....3
Ateliers 2 — Document API et Search Lite....5
2.1 Document API....5
2.2 Search Lite....5
Ateliers 3 — Mapping et indexation....6
3.1 Test des analyseurs....6
3.2 Mapping dynamique....6
3.3 Mapping explicite....7
3.4 Dynamic templates....8
3.5 ISM et Gabarit d'index....9

3.6 Réindexation et migration de mapping....13
Ateliers 4 : Ingestion de données....15
4.1 SDK Java · opensearch-java · Spring Boot....15
4.2 Ingest pipelines....23
4.3 Data Prepper (ingestion native OpenSearch)....25
4.4 Ingestion via KafkaConnect....29
Ateliers 5 — Requêtes DSL / Agrégations / Géoloc....32
5.1 Requêtes DSL....32
5.2 Agrégations....37
5.3 : Requêtes géospatiales....41
Ateliers 6 — OpenSearch Dashboards....45
6.1 Créer les Index Patterns....45
6.2 : Explorer avec Discover....46
6.3 : Créer des visualisations....48
6.4 : Assembler le tableau de bord....49
6.5 : Carte géospatiale (index position)....50
Ateliers 7 — Administration et sécurité....52
7.1 Gestion des nœuds....52
7.2 Snapshots....55
7.3 Activation de la sécurité TLS....56
Ateliers 8 — Supervision et monitoring du cluster....59
8.1 API _cat et diagnostics cluster....59
8.2 Query Insights (dashboard intégré)....61
8.3 Grafana....63

Ateliers 1 — Mise en place du cluster

1.1 Démarrage via docker compose

1.1.1 Premier démarrage

Vérifier votre espace disque libre (au moins 20 % de libre).

Récupérer le TPs/1_cluster/docker-compose.yml fourni :

```
docker compose up -d
```

Visualiser les logs de démarrage :

```
docker logs -f opensearch-dashboards
```

Accéder via un navigateur à :

- http://localhost:9200
- http://localhost:5601

Retrouver la Dev Tools Console.

Exécuter les requêtes suivantes :

API cat :

```
GET /_cat/nodes
GET /_cat/indices
```

Recherche de tous les documents :

```
GET /_search
```

1.2 Santé du cluster

Cluster health

- Exécuter :

```
GET /_cluster/health?pretty
```

- Combien de nœuds et shards sont disponibles ?
- Créer un index nommé blogs :

```
PUT /blogs
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1
  }
}
```

- Re-exécuter _cluster/health?pretty — couleur du statut ? Combien de shards disponibles, actifs ?

Fail-over

```
GET /_cat/nodes
```

Identifier le cluster_manager et arrêtez le :

```
docker stop opensearch-nodeX
```

Identifier le nouveau cluster manager

Regarder la santé du cluster

```
GET /_cluster/health
```

Regarder les index

```
GET /_cat/indices
```

Ateliers 2 — Document API et Search Lite

2.1 Document API

Indexer les 3 documents fournis via la commande curl ou via la Dev Tools Console :

```
curl -XPOST http://localhost:9200/blogs/_doc/ --data-binary '@entry1.json'
```

- Noter les IDs générés et récupérer un document via son ID :

```
GET /blogs/_doc/<id>
```

- Mettre à jour 1 document en ajoutant de nouveaux champs :
  - tags : Array
  - views : Initialisé à 0

```
POST /blogs/_update/<id>
{
  "doc": {
    "tags": ["opensearch", "formation"],
    "views": 0
  }
}
```

- Effectuer des mises à jour avec un script Painless :
  - Incrémenter un champ numérique
  - Ajouter un élément au tableau tags
  - Supprimer le champ tags

```
POST /blogs/_update/<id>
{
  "script": {
    "source": "ctx._source.views += 1"
  }
}
```

- Supprimer l'index et effectuer toutes les opérations en une seule commande Bulk API :

```
DELETE /blogs
```

2.2 Search Lite

Effectuer les recherches suivantes en utilisant la query string avec les paramètres :

- size, from
- q, df
- _source

```
GET /blogs/_search?q=titre:opensearch&size=5&from=0
GET /blogs/_search?q=opensearch&df=titre
GET /blogs/_search?_source=titre,auteur
```

Ateliers 3 — Mapping et indexation

3.1 Test des analyseurs

Tester les différences entre analyseur standard et analyseur français sur des textes en français avec

des requêtes REST.

Comment se comporte les mots avec accents, avec apostrophes, les mots composés ?

> ℹ️ Le Grok Debugger et l'outil Index Management dans OpenSearch Dashboards permettent de visualiser les mappings et tester les analyzers graphiquement.

3.2 Mapping dynamique

> *Objectif : Laisser OpenSearch inférer les types et constater les pièges.*

Indexer sans mapping préalable

Créer un index produits en envoyant directement un document :

```
POST /produits/_doc
{
  "nom": "OpenSearch t-shirt",
  "prix": "29.99",
  "stock": "150",
  "date_creation": "2024-01-15",
  "actif": "true"
}
```

Observer le mapping généré automatiquement :

```
GET /produits/_mapping
```

Constater les pièges

- Quel type a été inféré pour prix ? Pour stock ? Pour actif ?
- Tenter une agrégation sur le champ prix :

```
GET /produits/_search
{
  "aggs": {
    "prix_moyen": { "avg": { "field": "prix" } }
  }
}
```

- Que se passe-t-il ? Pourquoi ?
- Tenter un tri sur prix :

```
GET /produits/_search?sort=prix:asc
```

- Indexer un second document avec prix en nombre et observer le conflit :

```
POST /produits/_doc
{ "nom": "Mug OpenSearch", "prix": 12.5, "stock": 200 }
```

> ⚠️ **Conclusion attendue : le mapping dynamique est pratique mais dangereux en production. Tout champ mal inféré nécessitera une réindexation complète pour être corrigé.**

Utiliser me fichier bulk fourni pour importer 50 documents dans l'index :

```
curl -s -X POST "http://localhost:9200/_bulk" \
   -H "Content-Type: application/x-ndjson" \
   --data-binary @produits_bulk.ndjson
```

3.3 Mapping explicite

> *Objectif : contrôler exactement la structure avant d'ingérer.*

Créer un index avec mapping explicite

```
PUT /produits_v2
{
  "mappings": {
    "properties": {
      "nom": {
        "type": "text",
        "analyzer": "french",
        "fields": {
          "keyword": { "type": "keyword" }
        }
      },
      "prix":          { "type": "float" },
      "stock":         { "type": "integer" },
      "date_creation": { "type": "date", "format": "yyyy-MM-dd" },
      "actif":         { "type": "boolean" },
      "description":   { "type": "text", "analyzer": "french" },
      "tags":          { "type": "keyword" }
    }
  }
}
```

Indexer les mêmes documents et vérifier que les agrégations fonctionnent :

```
GET /produits_v2/_search
{
  "aggs": {
    "prix_moyen":  { "avg":   { "field": "prix" } },
    "stock_total": { "sum":   { "field": "stock" } },
    "par_tag":     { "terms": { "field": "tags" } }
  }
}
```

Explorer le multi-field

Le champ nom est à la fois text (recherche) et keyword (tri/agrégation) :

- Recherche full-text sur nom (utilise l'analyzer french) :

```
GET /produits_v2/_search?q=nom:t-shirt
```

- Tri alphabétique sur nom.keyword (valeur brute) :

```
GET /produits_v2/_search?sort=nom.keyword:asc
```

- Agrégation (facette) sur nom.keyword :

```
GET /produits_v2/_search
{
  "aggs": { "noms": { "terms": { "field": "nom.keyword" } } }
}
```

Tenter de modifier un type existant

Essayer de changer le type de prix en integer :

```
PUT /produits_v2/_mapping
```

```
{
  "properties": {
    "prix": { "type": "integer" }
  }
}
```

- Que retourne OpenSearch ? Pourquoi est-ce impossible ?
- Comment contourner ce problème en production ? (voir Atelier 3.6)

3.4 Dynamic templates

> *Objectif : Ggérer des structures de données variables sans mapper chaque champ manuellement.*

Cas réel : logs avec champs inconnus à l'avance

```
PUT /logs_app
{
  "mappings": {
    "dynamic_templates": [
      {
        "champs_id": {
          "match": "*_id",
          "mapping": { "type": "keyword" }
        }
      },
      {
        "champs_date": {
          "match": "*_at",
          "mapping": {
            "type": "date",
            "format": "epoch_millis||yyyy-MM-dd'T'HH:mm:ss"
          }
        }
      },
      {
        "strings_as_keyword": {
          "match_mapping_type": "string",
          "mapping": {
            "type": "text",
            "fields": {
              "keyword": { "type": "keyword", "ignore_above": 256 }
            }
          }
        }
      }
    ]
  }
}
```

Indexer un document de test :

```
POST /logs_app/_doc
{
  "user_id": "U-12345",
  "session_id": "abc-xyz",
  "created_at": "2024-03-15T10:30:00",
  "updated_at": 1710495000000,
  "message": "Connexion reussie",
  "niveau": "INFO"
}
```

- Vérifier le mapping appliqué :
- *user_id* et *session_id* sont-ils en keyword ?
- *created_at* et *updated_at* sont-ils en date ?

- *message* et *niveau* ont-ils bien le sous-champ ***.keyword*** ?
- Ajouter un champ *order_id* et vérifier qu'il est automatiquement mappé en keyword

3.5 ISM et Gabarit d'index

> ℹ️ *Objectif : mettre en place une politique de cycle de vie (ISM), un component template et un index template — puis valider le tout en injectant des documents via la Document API.*

Fil conducteur de cet atelier :

Étape	Objet	But
1	Politique ISM	Définir le cycle de vie : rollover → suppression
2	Component template	Réutiliser le mapping + analyzer de l'atelier 5.C
3	Index template	Associer politique ISM + component template
4	Index initial + alias	Point d'entrée pour les documents
5	Document API	Injecter des documents et observer le mapping appliqué
6	Vérifications	Contrôler analyzer, dynamic template, rollover ISM

Étape 1 — Créer la politique ISM

> ⚠️ *min_index_age: "1m" et "2m" (minutes) sont volontairement courts pour pouvoir observer le rollover pendant le TP. En production on utilise "1d" / "30d".*

Créer une politique *catalogue_policy* qui rollove après 5 documents ou 1 minute, et supprime après 2 minutes :

```
PUT _plugins/_ism/policies/catalogue_policy
{
  "policy": {
    "description": "Cycle de vie index catalogue produits",
    "default_state": "hot",
    "states": [
      {
        "name": "hot",
        "actions": [
          { "rollover": { "min_doc_count": 5, "min_index_age": "1m" } }
        ],
        "transitions": [
          { "state_name": "delete",
            "conditions": { "min_index_age": "2m" } }
        ]
      },
      {
        "name": "delete",
        "actions": [ { "delete": {} } ],
        "transitions": []
      }
    ]
  }
}
```

Vérifier la création :

```
GET _plugins/_ism/policies/catalogue_policy
```

Étape 2 — Créer le component template

On réutilise le mapping et l'analyzer ***french_sans_accents*** construits précédemment. Ce component template pourra être partagé avec d'autres index templates.

```
PUT _component_template/catalogue_mappings
{
  "template": {
    "settings": {
      "analysis": {
        "analyzer": {
          "french_sans_accents": {
            "type": "custom",
            "tokenizer": "standard",
            "filter": ["lowercase", "asciifolding", "french_stop",
"french_stemmer"]
          }
        },
        "filter": {
          "french_stop": { "type": "stop", "stopwords": "_french_" },
          "french_stemmer": { "type": "stemmer", "language": "french" }
        }
      }
    },
    "mappings": {
      "dynamic_templates": [
        { "ids_as_keyword": {
            "match": "*_id",
            "mapping": { "type": "keyword" }
        }},
        { "strings": {
            "match_mapping_type": "string",
            "mapping": {
              "type": "text",
              "fields": { "keyword": { "type": "keyword", "ignore_above": 256 } }
            }
        }}
      ],
      "properties": {
        "nom":        { "type": "text", "analyzer": "french_sans_accents",
                        "fields": { "keyword": { "type": "keyword" } } },
        "description":{ "type": "text", "analyzer": "french_sans_accents" },
        "prix":       { "type": "float" },
        "stock":      { "type": "integer" },
        "categorie":  { "type": "keyword" },
        "tags":       { "type": "keyword" },
        "date_ajout": { "type": "date", "format": "yyyy-MM-dd" },
        "actif":      { "type": "boolean" }
      }
    }
  }
}
```

Vérifier :

```
GET _component_template/catalogue_mappings
```

Étape 3 — Créer l'index template

L'index template associe le component template et la politique ISM à tout index dont le nom commence par catalogue- :

```
PUT _index_template/catalogue_template
{
  "index_patterns": ["catalogue-*"],
  "template": {
    "settings": {
      "number_of_shards": 2,
      "number_of_replicas": 1,
      "plugins.index_state_management.policy_id": "catalogue_policy",
      "plugins.index_state_management.rollover_alias": "catalogue"
    }
  },
  "composed_of": ["catalogue_mappings"],
  "priority": 500
}
```

Vérifier :

```
GET _index_template/catalogue_template
```

Étape 4 — Créer l'index initial et l'alias d'écriture

> *L'ISM a besoin d'un alias pour gérer le rollover. L'alias catalogue pointe sur catalogue-000001 en écriture.*

```
PUT /catalogue-000001
{
  "aliases": {
    "catalogue": {
      "is_write_index": true
    }
  }
}
```

L'ISM ne surveille pas les index en temps réel. Un job tourne en arrière-plan à intervalles réguliers pour détecter les nouveaux index à gérer. Par défaut ce délai est de **5 minutes**.

Vous pouvez forcer le refresh ISM via :

```
POST _plugins/_ism/add/catalogue-000001
{
  "policy_id": "catalogue_policy"
}
```

Vérifier que le template et l'ISM sont bien associés :

```
GET /catalogue-000001/_settings
GET _plugins/_ism/explain/catalogue-000001
```

- La politique catalogue_policy est-elle bien attachée à l'index ?
- Quel est l'état ISM actuel (hot) ?

Étape 5 — Injecter des documents via la BULK API

Indexer les 5 produits en une seule requête via l'alias catalogue :

```
POST /catalogue/_bulk
{ "index": {} }
{ "nom": "Clavier mecanique OpenSearch", "description": "Clavier ergonomique
ideal pour les developpeurs", "prix": 89.99, "stock": 42, "categorie":
"Informatique", "tags": ["clavier", "mecanique", "ergonomique"], "date_ajout":
"2026-01-10", "actif": true }
```

```
{ "index": {} }
{ "nom": "Souris sans fil silencieuse", "description": "Souris legere et
silencieuse pour bureaux open space", "prix": 34.50, "stock": 120, "categorie":
"Informatique", "tags": ["souris", "sans-fil", "silencieux"], "date_ajout":
"2026-01-12", "actif": true }
{ "index": {} }
{ "nom": "Ecran 27 pouces 4K", "description": "Moniteur haute resolution pour
graphistes et developpeurs", "prix": 459.00, "stock": 8, "categorie": "Ecrans",
"tags": ["ecran", "4K", "moniteur"], "date_ajout": "2026-01-15", "actif": true }
{ "index": {} }
{ "nom": "Casque audio bluetooth", "description": "Casque sans fil avec reduction
de bruit active", "prix": 149.00, "stock": 30, "categorie": "Audio", "tags":
["casque", "bluetooth", "sans-fil"], "date_ajout": "2026-01-18", "actif": true }
{ "index": {} }
{ "nom": "Webcam Full HD", "description": "Camera haute definition pour
visioconferences", "prix": 79.99, "stock": 65, "categorie": "Peripheriques",
"tags": ["webcam", "HD", "visioconference"], "date_ajout": "2026-01-20", "actif":
true }
```

Vérifier le nombre de documents et le mapping appliqué :

```
GET /catalogue/_count
GET /catalogue/_mapping
```

- Le mapping correspond-il à ce que définit le component template ?
- Les champs nom et description ont-ils bien l'analyzer *french_sans_accents* ?

Étape 6 — Vérifications

Tester le dynamic template

Ajouter un document avec un champ fournisseur_id — il doit être mappé en keyword grâce au template ids_as_keyword :

```
POST /catalogue/_doc
{
  "nom": "Hub USB-C 7 ports",
  "prix": 29.99, "stock": 55,
  "fournisseur_id": "FURN-0042",
  "categorie": "Accessoires",
  "actif": true
}
GET /catalogue/_mapping
```

- fournisseur_id est-il en keyword ?
- Pourquoi ? Quel dynamic template l'a capturé ?

Tester l'analyzer french_sans_accents

Les deux formes (singulier/pluriel, masculin/féminin, avec/sans accent) doivent retrouver les mêmes documents :

```
GET /catalogue/_search?q=description:ergonomiques
GET /catalogue/_search?q=description:ergonomique
GET /catalogue/_search?q=description:silencieuse
GET /catalogue/_search?q=description:silencieux
```

- Les deux formes de chaque paire retournent-elles le même résultat ?

Tester l'asciifolding — le document contenant 'développeurs' doit ressortir :

```
GET /catalogue/_search?q=description:développeur
```

- Le document avec 'developpeurs' (accent) ressort-il ? Pourquoi ?

Observer le rollover ISM

> *Le sweeper ISM tourne toutes les 5 minutes par défaut. Réduire l'intervalle pour ne pas attendre pendant le TP.*

Réduire d'abord l'intervalle du sweeper :

```
PUT _cluster/settings
{
  "persistent": {
    "plugins.jobscheduler.sweeper.period": "1m"
  }
}
```

Attendre 1-2 minutes, puis vérifier que l'ISM a bien pris en charge l'index :

```
GET _plugins/_ism/explain/catalogue-000001
```

- enabled: true est-il présent ?
- state.name vaut-il hot ?
- info.message indique-t-il Successfully initialized policy ?

```
GET /_cat/indices/catalogue-*?v
GET /_aliases?index=catalogue-*
```

- Un index catalogue-000002 a-t-il été créé ?
- Quel index est maintenant is_write_index: true ?
- L'alias catalogue pointe-t-il automatiquement sur le nouvel index en écriture ?

3.6 Réindexation et migration de mapping

> *Objectif : corriger un mauvais mapping en production sans perte de données ni coupure.*

Étape 1 — Créer le nouvel index avec le mapping corrigé

Repartir du mapping explicite de l'atelier 3.3, en ajoutant l'analyzer custom de l'atelier 3.5 :

```
PUT /produits_v3
{
  "settings": {
    "analysis": {
      "analyzer": {
        "french_sans_accents": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "asciifolding", "french_stop"]
        }
      },
      "filter": {
        "french_stop": { "type": "stop", "stopwords": "_french_" }
      }
    }
  },
  "mappings": {
    "properties": {
      "nom":          { "type": "text", "analyzer": "french_sans_accents",
                        "fields": { "keyword": { "type": "keyword" } } },
      "prix":         { "type": "float" },
      "stock":        { "type": "integer" },
      "date_creation":{ "type": "date", "format": "yyyy-MM-dd" },
      "actif":        { "type": "boolean" },
      "tags":         { "type": "keyword" }
    }
  }
```

```
}
```

Étape 2 — Réindexer les données

```
POST /_reindex
{
  "source": { "index": "produits" },
  "dest":   { "index": "produits_v3" }
}
```

- Vérifier que le nombre de documents est identique :

```
GET /produits/_count
GET /produits_v3/_count
```

Étape 3 — Basculer l'alias sans impact applicatif

L'alias permet de changer l'index cible sans modifier le code applicatif :

```
POST /_aliases
{
  "actions": [
    { "remove": { "index": "produits",    "alias": "catalogue" } },
    { "add":    { "index": "produits_v3", "alias": "catalogue" } }
  ]
}
```

- Vérifier que l'alias pointe bien sur le nouvel index :

```
GET /_aliases?pretty
```

- Effectuer une recherche via l'alias — les résultats sont identiques :

```
GET /catalogue/_search
```

- Supprimer l'ancien index une fois la migration validée :

```
DELETE /produits
```

Ateliers 4 : Ingestion de données

4.1 SDK Java · opensearch-java · Spring Boot

> *Objectif : ingérer des données dans OpenSearch depuis une application Java Spring Boot — connexion, indexation simple, BulkRequest, BulkIngester haute performance et gestion des erreurs.*

Résumé des étapes :

Étape	Objectif	Compétences travaillées
1	Création du projet Spring Boot	Spring Initializr, dépendance opensearch-java, configuration YAML
2	Connexion et santé du cluster	OpenSearchClient, RestClientTransport, test de connectivité
3	Indexation simple	Création d'index avec mapping explicite, CRUD documentaire
4	BulkRequest — batch manuel	Construction d'un batch, envoi, analyse de la réponse
5	BulkIngester — haute performance	Flush automatique, listener d'erreurs, thread-safety
6	Optimisations index	refresh_interval, replicas, translog avant ingestion massive

---

Étape 1 — Création du projet Spring Boot

Générer le projet via Spring Initializr (https://start.spring.io) avec les paramètres suivants :

- Group : org.formation
- Artifact : opensearch-ingestion
- Java : 21
- Dépendances à cocher : Spring Web, Spring Boot Actuator, Lombok

Ajouter manuellement la dépendance du client OpenSearch dans pom.xml :

```
<!-- pom.xml -->
<dependencies>

  <!-- Client OpenSearch officiel -->
  <dependency>
    <groupId>org.opensearch.client</groupId>
    <artifactId>opensearch-java</artifactId>
    <version>${opensearch-client.version}</version>
  </dependency>

  <!-- Transport HTTP (requis) -->
  <dependency>
    <groupId>org.apache.httpcomponents.client5</groupId>
    <artifactId>httpclient5</artifactId>
  </dependency>

  <!-- Sérialisation JSON -->
```

```
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
  </dependency>

</dependencies>
```

Configurer la connexion dans src/main/resources/application.yml :

```
opensearch:
  host: localhost
  port: 9200
  scheme: http       # https si sécurité activée
  # username: admin
  # password: admin

spring:
  application:
    name: opensearch-ingestion
```

Étape 2 — Connexion et santé du cluster

Créer la classe de configuration OpenSearch :

```
// src/main/java/com/formation/config/OpenSearchConfig.java
package com.formation.config;

import ...

@Configuration
public class OpenSearchConfig {

    @Value("${opensearch.host}")  private String host;
    @Value("${opensearch.port}")  private int    port;
    @Value("${opensearch.scheme}") private String scheme;

    @Bean
    public OpenSearchClient openSearchClient() {
          HttpHost host = new HttpHost(scheme, this.host, port);

          ApacheHttpClient5TransportBuilder builder =
                 ApacheHttpClient5TransportBuilder.builder(host);

          builder.setMapper(new JacksonJsonpMapper());

          return new OpenSearchClient(builder.build());

    }
}
```

Créer un service de vérification de la connexion :

```
// src/main/java/com/formation/service/ClusterService.java
@Service
@RequiredArgsConstructor
@Slf4j
public class ClusterService {

    private final OpenSearchClient client;

    public String checkHealth() throws IOException {
        var info = client.info();
        log.info("Connecté à OpenSearch {}",
            info.version().number());

        var health = client.cluster().health(h -> h);
        return "Cluster: " + health.clusterName()
             + " | Status: " + health.status()
```

```
                + " | Nodes: "  + health.numberOfNodes();
    }
}
```

Exposer via un endpoint REST et tester :

```
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class ClusterController {

    private final ClusterService clusterService;

    @GetMapping("/health")
    public ResponseEntity<String> health() throws IOException {
        return ResponseEntity.ok(clusterService.checkHealth());
    }
}

# Test
curl http://localhost:8080/api/health
# Attendu : Cluster: <nom> | Status: green | Nodes: 1
```

- La réponse contient-elle le nom de votre cluster et le statut ?
- Changer le port en 9999 dans application.yml — quelle exception obtient-on ?

Étape 3 — Indexation simple et CRUD documentaire

Créer la classe modèle Produit (avec Lombok) :

```
// src/main/java/com/formation/model/Produit.java
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Produit {
    private String  nom;
    private Double  prix;
    private Integer stock;
    private String  categorie;
    private String  dateAjout;  // format yyyy-MM-dd
    private Boolean actif;
}
```

Créer le service d'indexation avec les opérations CRUD :

```
// src/main/java/com/formation/service/ProduitService.java
@Service
@RequiredArgsConstructor
@Slf4j
public class ProduitService {

    private final OpenSearchClient client;
    private static final String INDEX = "produits-java";

    @PostConstruct public void init() throws IOException {
        createIndex();
    }

    // Créer l'index avec mapping explicite
    public void createIndex() throws IOException {
        boolean exists = client.indices()
            .exists(e -> e.index(INDEX)).value();
        if (exists) return;

        client.indices().create(c -> c
       .index(INDEX)
```

```
        .settings(s -> s
          .numberOfShards(3)       // 1 shard par nœud
          .numberOfReplicas(2)     // 2 réplicas = 3 copies au total
        ).mappings(m -> m
                .properties("nom",       p -> p.text(t -> t.analyzer("french")))
                .properties("prix",      p -> p.float_(f -> f))
                .properties("stock",     p -> p.integer(i -> i))
                .properties("categorie", p -> p.keyword(k -> k))
                .properties("dateAjout", p -> p.date(d -> d
                    .format("yyyy-MM-dd")))
                .properties("actif",     p -> p.boolean_(b -> b))
            )
        );
        log.info("Index {} créé", INDEX);
    }

    // Indexer un document
    public String indexer(Produit produit) throws IOException {
        var resp = client.index(i -> i
            .index(INDEX)
            .document(produit)
        );
        log.info("Indexé : {} ({})", resp.id(), resp.result());
        return resp.id();
    }

    // Récupérer par ID
    public Produit getById(String id) throws IOException {
        var resp = client.get(g -> g
            .index(INDEX).id(id), Produit.class);
        return resp.found() ? resp.source() : null;
    }

    // Supprimer par ID
    public void delete(String id) throws IOException {
        client.delete(d -> d.index(INDEX).id(id));
    }
}
```

Exposer via un controller REST et tester :

```
// POST /api/produits
curl -X POST http://localhost:8080/api/produits \
  -H 'Content-Type: application/json' \
  -d '{"nom":"Clavier mécanique","prix":89.99,
       "stock":42,"categorie":"Informatique",
       "dateAjout":"2026-01-10","actif":true}'

# Vérifier dans OpenSearch
GET /produits-java/_search
```

- L'ID généré est-il dans la réponse ?
- Récupérer le document par son ID : GET /api/produits/{id}
- Ré-indexer le même objet avec un prix différent — que vaut _version ?

Étape 4 — BulkRequest : batch manuel

Ajouter la méthode bulkIndex dans ProduitService pour envoyer un batch de documents :

```
// Dans ProduitService.java
public BulkStats bulkIndex(List<Produit> produits) throws IOException {

    // Construire la BulkRequest
    BulkRequest.Builder br = new BulkRequest.Builder();
    for (Produit p : produits) {
        br.operations(op -> op
```

```
            .index(i -> i
                .index(INDEX)
                .document(p)
            )
        );
    }

    BulkResponse resp = client.bulk(br.build());

    // Analyser la réponse item par item
    long errors = 0;
    if (resp.errors()) {
        for (BulkResponseItem item : resp.items()) {
            if (item.error() != null) {
                log.error("Erreur doc {}: {} ({})",
                    item.id(),
                    item.error().reason(),
                    item.error().type());
                errors++;
            }
        }
    }

    return new BulkStats(produits.size() - errors, errors,
                         resp.took());
}

@Data @AllArgsConstructor
public static class BulkStats {
    private long success, errors, tookMs;
}
```

Créer un endpoint de test qui génère 500 produits et les envoie en un batch :

```
// Dans ProduitController.java
@PostMapping("/bulk")
public ResponseEntity<ProduitService.BulkStats> bulk(
        @RequestParam(defaultValue="500") int count)
        throws IOException {

    List<Produit> produits = IntStream.range(0, count)
        .mapToObj(i -> Produit.builder()
            .nom("Produit Java " + String.format("%04d", i))
            .prix(Math.round(Math.random() * 490 + 10) / 1.0)
            .stock((int)(Math.random() * 200))
            .categorie(List.of(
                "Informatique","Audio","Écrans","Accessoires"
            ).get(i % 4))
            .dateAjout("2026-01-15")
            .actif(true)
            .build())
        .collect(Collectors.toList());

    var stats = produitService.bulkIndex(produits);
    return ResponseEntity.ok(stats);
}

# Test
curl -X POST 'http://localhost:8080/api/produits/bulk?count=500'
```

- Combien de ms a pris le bulk (champ tookMs) ?
- GET /produits-java/_count — le total est-il correct ?

> ⚠️ *La Bulk API retourne HTTP 200 même si certains documents ont échoué. Le champ errors:true dans la réponse est le seul indicateur — ne jamais ignorer l'analyse des items.*

Étape 5 — BulkIngester : haute performance avec flush automatique

Le BulkIngester est la solution recommandée pour les applications long-running — il gère automatiquement le flush par taille, par nombre de documents ou par intervalle de temps.

```
// Dans ProduitService.java — méthode haute performance
public BulkStats ingesterBulk(List<Produit> produits)
        throws IOException, InterruptedException {

    AtomicLong success = new AtomicLong(0);
    AtomicLong errors  = new AtomicLong(0);

    // Configurer le BulkIngester
    try (BulkIngester<Produit> ingester = BulkIngester.of(b -> b
        .client(client)
        .maxOperations(500)           // flush à 500 opérations
        .maxSize(10 * 1024 * 1024L)   // ou 10 Mo
        .flushInterval(5, TimeUnit.SECONDS)  // ou toutes les 5s
        .listener(new BulkListener<Produit>() {

            @Override
            public void beforeBulk(long executionId,
                BulkRequest request,
                List<Produit> contexts) {
                log.debug("Bulk {} : {} docs",
                    executionId, request.operations().size());
            }

            @Override
            public void afterBulk(long executionId,
                BulkRequest request,
                List<Produit> contexts,
                BulkResponse response) {
                // Compter succès et erreurs
                response.items().forEach(item -> {
                    if (item.error() != null) {
                        errors.incrementAndGet();
                        log.error("Erreur: {}",
                            item.error().reason());
                    } else {
                        success.incrementAndGet();
                    }
                });
            }

            @Override
            public void afterBulk(long executionId,
                BulkRequest request,
                List<Produit> contexts,
                Throwable failure) {
                log.error("Bulk {} échoué: {}",
                    executionId, failure.getMessage());
                errors.addAndGet(request.operations().size());
            }
        })
    )) {
        // Ajouter tous les documents — flush géré automatiquement
        for (Produit p : produits) {
            ingester.add(op -> op
                .index(i -> i.index(INDEX).document(p)),
                p   // contexte passé au listener
            );
        }
    } // close() = flush final garanti

    return new BulkStats(success.get(), errors.get(), 0);
}
```

Créer un endpoint de benchmark pour comparer BulkRequest vs BulkIngester :

```
// GET /api/produits/benchmark?count=10000
@GetMapping("/produits/benchmark")
public ResponseEntity<Map<String,Object>> benchmark(
        @RequestParam(defaultValue="10000") int count)
        throws IOException, InterruptedException {

    List<Produit> produits = generateProduits(count);

    // Test BulkRequest
    long t1 = System.currentTimeMillis();
    var stats1 = produitService.bulkIndex(produits);
    long duration1 = System.currentTimeMillis() - t1;

    // Test BulkIngester
    long t2 = System.currentTimeMillis();
    var stats2 = produitService.ingesterBulk(produits);
    long duration2 = System.currentTimeMillis() - t2;

    return ResponseEntity.ok(Map.of(
        "count",        count,
        "bulkRequest",  Map.of("ms", duration1,
            "docsPerSec", count * 1000 / duration1),
        "bulkIngester", Map.of("ms", duration2,
            "docsPerSec", count * 1000 / duration2)
    ));
}

# Test
curl 'http://localhost:8080/api/produits/benchmark?count=10000'
```

- Quel est le débit (docs/s) de chaque approche ?
- Modifier maxOperations à 100 puis à 2000 — quel impact sur le débit ?
- Modifier flushInterval à 1s — que se passe-t-il si le producteur s'arrête entre deux flush ?

> *Le BulkIngester est thread-safe — une seule instance partagée dans un bean Spring @Service suffit pour toute l'application. Ne pas en créer un par requête.*

Étape 6 — Optimisations index avant ingestion massive

Implémenter un service d'optimisation qui désactive le refresh et les réplicas avant ingestion :

```
// Dans ProduitService.java
public void setIngestionMode(boolean enabled) throws IOException {
    String refreshInterval = enabled ? "-1" : "1s";
    int    replicas        = enabled ?   0  :   1 ;
    String durability      = enabled ? "async" : "request";

    client.indices().putSettings(s -> s
        .index(INDEX)
        .settings(settings -> settings
            .refreshInterval(t -> t.time(refreshInterval))
            .numberOfReplicas(String.valueOf(replicas))
        )
    );
    log.info("Mode ingestion {} : refresh={}, replicas={}",
        enabled ? "ON" : "OFF", refreshInterval, replicas);
}

public void forceRefresh() throws IOException {
    client.indices().refresh(r -> r.index(INDEX));
    log.info("Refresh forcé sur {}", INDEX);
}
```

Créer un endpoint qui enchaîne les 3 phases — avant / ingestion / après :

```
// POST /api/produits/massive?count=50000
```

```
@PostMapping("/massive")
public ResponseEntity<Map<String,Object>> massive(
        @RequestParam(defaultValue="50000") int count)
        throws IOException, InterruptedException {

    // AVANT : mode ingestion
    produitService.setIngestionMode(true);

    long start = System.currentTimeMillis();
    var stats = produitService.ingesterBulk(
        generateProduits(count));
    long elapsed = System.currentTimeMillis() - start;

    // APRÈS : restaurer et rafraîchir
    produitService.setIngestionMode(false);
    produitService.forceRefresh();

    return ResponseEntity.ok(Map.of(
        "total",   count,
        "success", stats.getSuccess(),
        "errors",  stats.getErrors(),
        "ms",      elapsed,
        "docsPerSec", count * 1000 / elapsed
    ));
}

# Test
curl -X POST 'http://localhost:8080/api/produits/massive?count=50000'

# Vérifier le count final
GET /produits-java/_count
```

- Comparer le débit avec et sans setIngestionMode(true)
- Pendant l'ingestion, exécuter GET /produits-java/_count à plusieurs reprises — que remarquez-vous avec refresh_interval: -1 ?
- Après setIngestionMode(false) et forceRefresh(), le count est-il correct ?

> *Sur un cluster mono-nœud (configuration TP), number_of_replicas: 0 n'a pas d'impact visible. L'effet sur le débit est maximal sur un cluster multi-nœuds où chaque réplica reçoit une copie réseau de chaque document.*

Créer une méthode permettant d'ajouter de la diversité dans les produits créés (utilisés dans les ateliers suivants)

```
private List<Produit> generateProduits(int count) {
    Random random = new Random();

    List<Produit> produits = IntStream.range(0, count)
            .mapToObj(i -> Produit.builder()
                    .nom(List.of(
                            "Produit Informatique "+ String.format("%04d",
i),"Audio","Clavier numérique " + String.format("%04d", i),"Produit numérique " +
String.format("%04d", i)
                    ).get(random.nextInt(4) % 4))
                    .prix(Math.round(Math.random() * 490 + 10) / 1.0)
                    .stock((int)(Math.random() * 200))
                    .categorie(List.of(
                            "Informatique","Audio","Écrans","Accessoires"
                    ).get(random.nextInt(4) % 4))
                    .dateAjout("2026-01-"+random.nextInt(31))
                    .actif(true)
```

```
                    .build())
            .collect(Collectors.toList());
    return produits;
}
```

Références

- Documentation opensearch-java : https://opensearch.org/docs/latest/clients/java/
- Javadoc BulkIngester : https://javadoc.io/doc/org.opensearch.client/opensearch-java
- Spring Boot + OpenSearch : https://opensearch.org/docs/latest/clients/java-integrations/

4.2 Ingest pipelines

Objectif : Utiliser une pipeline pour convertir correctement les données d'entrée.

Mapping des transfomations à effectuer

Champ	Type actuel (string)	Type cible
prix	"29.99" (string)	**float**
stock	"150" (string)	**integer**
actif	"true" (string)	**boolean**
date_creation	"2024-01-15" (string)	**date (ISO 8601)**

4.2.1 Création de la pipeline

```
PUT /_ingest/pipeline/produits-typing
{
  "description": "Convertit les types de champs de l'index produits",
  "processors": [
    {
      "convert": {
        "field": "prix",
        "type": "float"
      }
    },
    {
      "convert": {
        "field": "stock",
        "type": "integer"
      }
    },
    {
      "convert": {
        "field": "actif",
        "type": "boolean"
      }
    },
    {
      "date": {
        "field": "date_creation",
        "formats": [
          "yyyy-MM-dd"
```

```
        ],
        "target_field": "date_creation",
        "timezone": "Europe/Paris"
      }
    }
  ]
}
```

Vérifier que le pipeline a été créé

```
GET /_ingest/pipeline/produits-typing
```

4.2.2 Tester le pipeline sans indexer

Exécutez la simulation avec un document fictif :

```
POST /_ingest/pipeline/produits-typing/_simulate
{
  "docs": [
    {
      "_source": {
        "nom": "Produit test",
        "prix": "49.99",
        "stock": "25",
        "actif": "false",
        "date_creation": "2024-06-15"
      }
    }
  ]
}
```

Vérifiez dans la réponse que les types ont bien été convertis :

Tester avec une valeur invalide

```
POST /_ingest/pipeline/produits-typing/_simulate
{
  "docs": [
    {
      "_source": {
        "prix": "prix_inconnu",
        "stock": "10",
        "actif": "true",
        "date_creation": "2024-01-01"
      }
    }
  ]
}
```

Appliquer le pipeline aux données existantes

Créer l'index cible avec le bon mapping

```
PUT /produits-v2
{
  "settings": {
    "number_of_shards":   1,
    "number_of_replicas": 0
  },
  "mappings": {
    "properties": {
      "nom":  {
        "type": "text",
        "fields": {
          "keyword": {
          "type": "keyword"
          }
        }
      },
      "prix": {
        "type": "float"
      },
      "stock": {
        "type": "integer"
      },
      "actif": {
        "type": "boolean"
      },
      "date_creation": {
        "type": "date"
      }
    }
  }
}
```

Réindexez depuis produits vers produits-v2 en appliquant le pipeline :

```
POST /_reindex
{
  "source": {
    "index": "produits"
  },
  "dest": {
    "index":"produits-v2",
    "pipeline": "produits-typing"
  }
}
```

- Vérifiez les documents réindexés :

```
GET /produits-v2/_count
GET /produits-v2/_mapping
GET /produits-v2/_search
```

4.3 Data Prepper (ingestion native OpenSearch)

> *Objectif : instrumenter une application Spring Boot avec l'agent OTel, envoyer les traces vers Data Prepper, et les visualiser dans OpenSearch Dashboards Trace Analytics.*

Fil conducteur de l'atelier :

Étape	Objectif	Ce qu'on observe
1	Configurer Data Prepper	Pipeline otel-trace validé, GET /list confirme le démarrage
2	Créer l'application Java	Spring Boot + REST API simple avec 3 endpoints imbriqués
3	Instrumenter avec OTel	Java Agent attaché, variables d'env OTLP configurées
4	Générer des traces	curl sur les endpoints → spans visibles dans OpenSearch
5	Trace Analytics	Service Map, timeline de trace, latences par service
6	Traces d'erreur	Exception simulée → trace rouge dans le dashboard

---

Étape 1 — Configurer le pipeline Data Prepper

Récupérer le doker compose fourni :

- Le regarder et le démarrer:
- Visualiser également le fichiers et en particulier la pipeline

Explication de la pipeline :

- `otel-trace-pipeline` reçoit les traces via gRPC (port 21890) et les distribue en **fork** vers deux pipelines en parallèle — c'est le point d'entrée et le distributeur.
- `raw-trace-pipeline` prend chaque span individuel, le transforme au format OpenSearch APM et l'indexe dans `otel-v1-apm-span-*` — c'est ce qui alimente la timeline et la vue détail des traces.
- `service-map-pipeline` analyse les relations parent/enfant entre spans pour calculer le graphe des dépendances inter-services et l'indexe dans `otel-v1-apm-service-map` — c'est ce qui alimente la Service Map.

Démarrer la stack et vérifier que Data Prepper est opérationnel :

```
docker compose up -d

# Attendre ~30s puis vérifier les pipelines
curl http://localhost:4900/list
# Attendu : liste des 3 pipelines actifs

# Vérifier les index OTel dans OpenSearch (vides pour l'instant)
GET /_cat/indices/otel*?v
```

- Les 3 pipelines apparaissent-ils dans /list ?
- Aucune erreur dans les logs : docker logs data-prepper --tail 20

Étape 2 : Récupérer l'application Java Spring Boot

> *On crée une petite API de gestion de commandes — 3 services simulés (api-gateway, order-service, inventory-service) dans une seule application pour observer les spans imbriqués.*

Le code fourni 'est une API REST de gestion de commandes avec trois endpoints :

- **lister les commandes** (`GET /api/orders`)
- **créer une commande** (`POST /api/orders`) qui simule un appel interne à un service d'inventaire
- **récupérer une commande par ID** (`GET /api/orders/{id}`).

Chaque endpoint introduit une **latence aléatoire** pour simuler un vrai comportement réseau, et certains appels déclenchent des **erreurs contrôlées** — stock insuffisant si `productId=OUT-OF-STOCK`, commande introuvable si `id=error`.

Récupérer le code, le compiler et vérifier que l'application démarre :

```
mvn clean package -q
java -jar target/otel-demo-*.jar
# Tester : curl http://localhost:8081/api/orders
```

Étape 3 — Attacher le Java Agent OpenTelemetry

Visualiser le fichier start.sh fourni à la racine du projet.

Il démarre l'application avec l'agent OpenTelemetry attaché et sa configuration

> *L'agent instrumente automatiquement Spring Web (chaque requête HTTP = 1 span), RestTemplate, JDBC, et des dizaines d'autres frameworks. Aucune annotation @Span ni import OTel dans le code.*

Vérifier que l'agent se connecte — dans les logs de démarrage

Étape 4 — Générer des traces

Envoyer plusieurs requêtes pour créer des traces variées :

Requêtes normales

```
# Lister les commandes (1 span simple)
curl http://localhost:8081/api/orders

# Créer une commande (spans imbriqués : HTTP → checkInventory)
curl -X POST http://localhost:8081/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"productId": "PROD-001", "quantity": 2}'

# Détail d'une commande
curl http://localhost:8081/api/orders/ORD-123

# Répéter plusieurs fois pour avoir des données
for i in {1..10}; do
  curl -s http://localhost:8081/api/orders > /dev/null
  curl -s -X POST http://localhost:8080/api/orders \
    -H 'Content-Type: application/json' \
    -d '{"productId": "PROD-00'$i'", "quantity": '$i'}' > /dev/null
done
```

Traces d'erreur (étape 6)

```
# Erreur 404 — commande introuvable
curl http://localhost:8081/api/orders/error

# Erreur 409 — stock insuffisant
curl -X POST http://localhost:8081/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"productId": "OUT-OF-STOCK", "quantity": 99}'
```

Vérifier que les index OTel sont alimentés dans OpenSearch :

```
GET /_cat/indices/otel*?v
```

```
# Attendu :
# otel-v1-apm-span-YYYY.MM.DD
# otel-v1-apm-service-map

GET /otel-v1-apm-span-*/_count
# Vérifier que le count augmente après chaque requête

# Explorer un span
GET /otel-v1-apm-span-*/_search
{
  "size": 1,
  "_source": ["traceId","spanId","name",
              "durationInNanos","status"]
}
```

- Quels champs contient un span ? (traceId, parentSpanId, durationInNanos…)
- Les spans d'erreur ont-ils un champ status différent ?

Étape 5 — Visualiser dans OpenSearch Dashboards

Accéder à Trace Analytics

Dans OpenSearch Dashboards (http://localhost:5601) :

- Menu hamburger → Observability → Trace Analytics
- Si l'interface est vide : attendre ~30s que Data Prepper popule les index

Service Map

- Cliquer sur l'onglet Service Map
- Le service order-service doit apparaître comme nœud
- Les edges représentent les appels entre spans (ici appels internes simulés)
- Cliquer sur order-service → observer la latence moyenne et le taux d'erreur

Explorer une trace

- Onglet Traces → liste des traces récentes
- Cliquer sur une trace pour voir la timeline Gantt des spans
- Identifier la trace avec le plus de spans (requête POST createOrder)
- Observer la hiérarchie : span HTTP → span checkInventory
- Comparer les durées — quel span prend le plus de temps ?

Filtrer par service et status

```
# Dans Dev Tools — chercher les spans lents (> 100ms)
GET /otel-v1-apm-span-*/_search
{
  "query": {
    "range": {
      "durationInNanos": { "gte": 100000000 }
    }
  },
  "sort": [{ "durationInNanos": "desc" }],
  "_source": ["name","durationInNanos",
              "traceId","status"],
```

```
  "size": 5
}
```

- Quels sont les 5 spans les plus lents ?
- Correspondent-ils aux appels checkInventory (latence simulée 30-100ms) ?

Étape 6 — Observer les traces d'erreur

Générer plusieurs erreurs et les observer dans Trace Analytics :

```
# Générer 5 erreurs de types différents
for i in {1..5}; do
  curl -s http://localhost:8080/api/orders/error > /dev/null
done
for i in {1..3}; do
  curl -s -X POST http://localhost:8080/api/orders \
    -H 'Content-Type: application/json' \
    -d '{"productId":"OUT-OF-STOCK","quantity":1}' > /dev/null
done
```

Dans Trace Analytics → Traces :

- Les traces en erreur apparaissent en rouge
- Cliquer sur une trace en erreur → le span en erreur est mis en évidence
- Observer le champ status.message qui contient le message d'exception

Chercher les spans en erreur dans Dev Tools :

```
GET /otel-v1-apm-span-*/_search
{
  "query": {
    "term": { "status": "STATUS_CODE_ERROR" }
  },
  "_source": ["name","traceId","status",
              "statusMessage","durationInNanos"],
  "size": 10
}
```

- Quel est le statusMessage des erreurs OUT_OF_STOCK vs NOT_FOUND ?
- La durée des spans en erreur est-elle différente des spans normaux ?

> *Le Java Agent OTel capture automatiquement les exceptions non catchées ET les exceptions catchées annotées avec @WithSpan. Les exceptions catchées dans le @RestControllerAdvice sont enregistrées comme STATUS_CODE_ERROR dans le span.*

4.4 Ingestion via KafkaConnect

> *Objectif : Déverser un topic kafka vers un index OpenSearch.*

4.3.1 Installation du plugin OpenSearch dans kafka connect

Visualiser le fichier Dockerfile fourni et construire une image my-kafka-connect-with-opensearch via :

```
docker build -t my-kafka-connect-with-opensearch .
```

4.3.2 Démarrage de la stack

Démarrer la nouvelle stack qui ajoute l'image précédemment construite, ElasticSearch et Kibana

```
docker-compose up -d
```

Vérifier l'installation du plugin via :

```
http://localhost:8083/connector-plugins
```

Ou via la console redpanda :

```
http://localhost:9090
```

4.3.3 Alimenter un topic position

Utiliser le programme fourni pour alimenter un topic ***position***.
Le programme simule une flotte de coursier qui envoient régulièrement leur position dans le topic.

```
./mvnw clean package
```

Le programme prend 4 arguments :

- Le nombre de coursiers
- Le nombre de message envoyés par chaque coursier
- Le temps de pause entre chaque message en ms
- Le type d'envoi :
  - 0=fire_and_forget
  - 1=Envoi synchrone
  - 2=Envoi avec callback

Générer 500 000 messages dans le topic avec :

```
java -jar target/producer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 100 5000 100 2
```

Vérifier les messages dans la console redpanda

4.3.4 Préparation de l'index

Se connecter au dashboard localhost :5601 et dans la DevConsole exécuter :

```
PUT /position
{
  "mappings":{
    "properties": {
      "@timestamp": {
       "type": "date",
       "format": "epoch_millis"
       },
       "currentPosition": { "type": "geo_point" }
     }
  }
}
```

La commande crée un index position avec pour l'instant un seul champ *@timestamp*

4.3.5 Création du connecteur

Définir un connecteur ***opensearch-sink*** qui déverse le topic position dans l'index position :

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
 "name": "opensearch-sink",
  "config": {
    "connector.class":
      "io.aiven.kafka.connect.opensearch.OpensearchSinkConnector",
      "tasks.max": "1",
      "topics": "position",
      "topic.index.map": "position:position_index",
      "connection.url": "http://opensearch-node1:9200",
      "key.ignore": "true",
```

```
      "schema.ignore": "true",
      "key.converter": "org.apache.kafka.connect.json.JsonConverter",
      "key.converter.schemas.enable": "false",
      "value.converter": "org.apache.kafka.connect.json.JsonConverter",
      "value.converter.schemas.enable": "false"
  }
}'
```

Vous pouvez vérifier la bonne installation du connecteur :

- Via les logs de KafkaConnect
- Via http://localhost:8083 /connectors : Liste des connecteurs actifs
- Via l'interface de RedPanda

Alimenter le topic position
Visualiser les effets du connecteur via la DevConsole
Améliorer le fichier de configuration afin que le timestamp Kafka soit renseigné
dans le champ *@timestamp* de l'index *position* d'OpenSearch (nécessaire pour la suite et Open Dashboards)

Ateliers 5 — Requêtes DSL / Agrégations / Géoloc

5.1 Requêtes DSL

> *Objectif : maîtriser le Query DSL d'OpenSearch — requêtes full-text, filtres, recherches exactes et combinaisons booléennes sur l'index produits créé en atelier 4.1.*

5.1.1 Préparation — Vérifier l'index produits

L'index produits-java doit contenir des données issues de l'atelier 4.1. Vérifier :

```
GET /produits-java/_count
# Attendu : count > 0

GET /produits-java/_mapping
# Vérifier la présence de : nom (text), prix (float),
# categorie (keyword), stock (integer), actif (boolean)
```

- Si l'index est vide, relancer l'application spring-boot et l'endpoint /api/produits/bulk?count=1000 de l'atelier 4.1

5.1.2 Requêtes full-text (match)

La requête match analyse la chaîne de recherche avec l'analyzeur du champ avant de chercher.

Recherche simple sur le nom

```
GET /produits-java/_search
{
  "query": {
    "match": {
      "nom": "produit 0000"
    }
  }
}
```

- Combien de résultats ? Les deux mots sont-ils tous les deux présents dans chaque résultat ?
- Par défaut match utilise l'opérateur OR — essayer avec l'opérateur AND :

```
GET /produits-java/_search
{
  "query": {
    "match": {
      "nom": {
        "query": "produit 0000",
        "operator": "and"
      }
    }
  }
}
```

- Quelle différence observez-vous dans les résultats ?

match_phrase — ordre et proximité

match_phrase exige que les termes apparaissent dans le même ordre et consécutifs :

```
GET /produits-java/_search
```

```
{
  "query": {
    "match_phrase": {
      "nom": "Produit Java"
    }
  }
}
```

- Comparer avec match — y a-t-il des résultats supplémentaires avec match ?

multi_match — chercher sur plusieurs champs

```
GET /produits-java/_search
{
  "query": {
    "multi_match": {
      "query": "Informatique",
      "fields": ["nom", "categorie"]
    }
  }
}
```

- Booster le champ nom (score x2) en ajoutant "fields": ["nom^2", "categorie"]

> *L'analyse du texte est appliquée sur les champs de type text. Sur un champ keyword (comme categorie), la recherche est exacte — majuscules et minuscules comptent.*

5.1.3 Filtres — term, terms, range

Les filtres ne calculent pas de score — ils sont plus rapides que les requêtes full-text et sont mis en cache par OpenSearch.

term — valeur exacte

```
GET /produits-java/_search
{
  "query": {
    "term": {
      "categorie": "Informatique"
    }
  }
}
```

- Essayer avec "informatique" (minuscule) — que se passe-t-il ? Pourquoi ?
- Essayer avec le champ nom — que se passe-t-il ? (nom est text, pas keyword)

terms — plusieurs valeurs

```
GET /produits-java/_search
{
  "query": {
    "terms": {
      "categorie": ["Informatique", "Audio", "Écrans"]
    }
  }
}
```

- Combien de catégories différentes existent ? Utiliser une agrégation terms (atelier 6)

range — plage de valeurs

```
GET /produits-java/_search
{
  "query": {
    "range": {
      "prix": {
        "gte": 50,
        "lte": 200
      }
    }
  }
}
```

- Trouver les produits avec un stock critique (stock < 10)
- Trouver les produits dont le prix est dans le top 10% (prix > 450)

5.1.4 Requêtes composées — bool

La requête bool combine plusieurs clauses : must (obligatoire + score), filter (obligatoire sans score), should (bonus de score), must_not (exclusion).

must + filter

Chercher les produits Informatique entre 50 et 300€ dont le nom contient 'Pro' :

```
GET /produits-java/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "nom": "Produit" } }
      ],
      "filter": [
        { "term":  { "categorie": "Informatique" } },
        { "range": { "prix": { "gte": 50, "lte": 300 } } }
      ]
    }
  }
}
```

- Pourquoi mettre categorie et range dans filter plutôt que must ?

should — boost de score

Chercher les produits Informatique, en boostant ceux qui ont 'Pro' ou 'Premium' dans le nom :

```
GET /produits-java/_search
{
  "query": {
    "bool": {
      "filter": [
        { "term": { "categorie": "Informatique" } }
      ],
      "should": [
        { "match": { "nom": "Pro" } },
        { "match": { "nom": "Premium" } }
      ]
    }
  }
}
```

- Les produits sans 'Pro' ni 'Premium' apparaissent-ils quand même ? Pourquoi ?

must_not — exclusion

```
GET /produits-java/_search
{
  "query": {
    "bool": {
      "filter": [
        { "term": { "actif": true } }
      ],
      "must_not": [
        { "term": { "categorie": "Accessoires" } },
        { "range": { "stock": { "lte": 0 } } }
      ]
    }
  }
}
```

- Que retourne cette requête en une phrase ?

5.1.5 Pagination et tri

from / size

```
GET /produits-java/_search
{
  "from": 0,
  "size": 10,
  "query": { "match_all": {} },
  "sort": [
    { "prix": { "order": "desc" } }
  ]
}
```

- Récupérer la 2e page (documents 11 à 20) — modifier from
- Trier par categorie ASC puis prix DESC — ajouter un second critère dans sort

> ⚠️ *Pour plus de 10 000 résultats, from+size est bloqué par index.max_result_window. Utiliser search_after ou le Scroll API.*

_source filtering — projections

Ne récupérer que les champs utiles pour réduire la bande passante :

```
GET /produits-java/_search
{
  "_source": ["nom", "prix", "categorie"],
  "query": { "match_all": {} },
  "size": 5
}
```

- Exclure le champ dateAjout avec "_source": { "excludes": ["dateAjout"] }

5.1.6 Highlight — mise en évidence des termes

```
GET /produits-java/_search
{
  "query": {
    "match": { "nom": "Produit" }
  },
  "highlight": {
    "fields": {
      "nom": {}
    }
```

```
  }
}
```

- Les résultats contiennent un champ highlight.nom — observer les balises
- Personnaliser les balises : ajouter "pre_tags": ["**"], "post_tags": [""] dans highlight

5.2 Agrégations

> *Objectif : exploiter les agrégations OpenSearch pour analyser l'index produits — statistiques, distributions, métriques et imbrications.*

5.2.1 Agrégations de métriques

Les agrégations de métriques calculent une valeur unique sur un ensemble de documents.

Statistiques globales sur le prix

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "prix_stats": {
      "stats": { "field": "prix" }
    }
  }
}
```

- Quels sont min, max, avg et sum des prix ?
- Remplacer stats par extended_stats — quels champs supplémentaires apparaissent ?

> *size: 0 évite de retourner les documents source — on ne s'intéresse qu'aux agrégations.*

Percentiles

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "prix_percentiles": {
      "percentiles": {
        "field": "prix",
        "percents": [25, 50, 75, 90, 95, 99]
      }
    }
  }
}
```

- Quelle est la valeur au 95e percentile ? Combien de produits coûtent plus cher ?

Cardinality — valeurs distinctes

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "nb_categories": {
      "cardinality": { "field": "categorie" }
    }
  }
}
```

- Combien de catégories distinctes existent dans l'index ?

5.2.2 Agrégations de buckets

Les agrégations de buckets répartissent les documents en groupes (buckets).

terms — distribution par catégorie

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "par_categorie": {
      "terms": {
        "field": "categorie",
        "size": 10
      }
    }
  }
}
```

- Quelle catégorie contient le plus de produits ?
- Que représente doc_count_error_upper_bound ?
- Trier par doc_count ASC : ajouter "order": { "_count": "asc" }

histogram — distribution des prix

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "distribution_prix": {
      "histogram": {
        "field": "prix",
        "interval": 50
      }
    }
  }
}
```

- Quelle tranche de prix concentre le plus de produits ?
- Réduire l'intervalle à 25 — l'histogramme est-il plus précis ?

range — tranches personnalisées

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "tranches_prix": {
      "range": {
        "field": "prix",
        "ranges": [
          { "key": "Entrée de gamme", "to": 50 },
          { "key": "Milieu de gamme", "from": 50, "to": 200 },
          { "key": "Haut de gamme",   "from": 200 }
        ]
      }
    }
  }
}
```

- Combien de produits haut de gamme (> 200€) y a-t-il ?

5.2.3 Agrégations imbriquées

Les agrégations peuvent être imbriquées — chaque bucket peut contenir ses propres sous-agrégations.

Prix moyen par catégorie

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "par_categorie": {
      "terms": { "field": "categorie" },
      "aggs": {
        "prix_moyen": {
          "avg": { "field": "prix" }
        },
        "stock_total": {
          "sum": { "field": "stock" }
        }
      }
    }
  }
}
```

- Quelle catégorie a le prix moyen le plus élevé ?
- Quelle catégorie a le stock total le plus important ?

Trier les buckets par sous-agrégation

Trier les catégories par prix moyen décroissant :

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "par_categorie": {
      "terms": {
        "field": "categorie",
        "order": { "prix_moyen": "desc" }
      },
      "aggs": {
        "prix_moyen": { "avg": { "field": "prix" } }
      }
    }
  }
}
```

histogram + stats imbriquées

Distribution des prix par tranche de 100€ avec statistiques de stock par tranche :

```
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "tranches": {
      "histogram": {
        "field": "prix",
        "interval": 100,
        "min_doc_count": 1
      },
      "aggs": {
        "stock_stats": {
          "stats": { "field": "stock" }
        }
      }
    }
```

```
  }
}
```

- Les produits les plus chers ont-ils tendance à avoir un stock plus faible ?

5.2.4 Combiner requête et agrégation

Une agrégation s'applique sur les documents retournés par la requête — pas sur tous les documents de l'index.

Statistiques sur les produits actifs uniquement

```
GET /produits-java/_search
{
  "size": 0,
  "query": {
    "term": { "actif": true }
  },
  "aggs": {
    "par_categorie": {
      "terms": { "field": "categorie" },
      "aggs": {
        "prix_moyen":  { "avg": { "field": "prix" } },
        "prix_max":    { "max": { "field": "prix" } }
      }
    }
  }
}
```

- Comparer les prix moyens avec et sans le filtre actif:true

global — agrégation sur tous les docs indépendamment de la requête

```
GET /produits-java/_search
{
  "size": 0,
  "query": {
    "term": { "categorie": "Informatique" }
  },
  "aggs": {
    "informatique_avg": {
      "avg": { "field": "prix" }
    },
    "global_avg": {
      "global": {},
      "aggs": {
        "prix_global": { "avg": { "field": "prix" } }
      }
    }
  }
}
```

- Le prix moyen Informatique est-il au-dessus ou en-dessous de la moyenne globale ?

5.3 : Requêtes géospatiales

> *Objectif : exploiter les capacités géospatiales d'OpenSearch sur l'index position alimenté par Kafka Connect en atelier 4.3 — recherche par proximité, zone géographique et agrégations.*

5.3.1 Vérification de l'index position

L'index position doit contenir les messages du topic Kafka générés par la flotte de coursiers.

```
GET /position/_count
# Attendu : 500 000 documents environ

GET /position/_mapping
# Vérifier la présence du champ position de type geo_point

# Explorer un document
GET /position/_search
{
  "size": 1,
  "_source": true
}
```

- Quels champs contient un document de position ?
- Le champ position contient-il lat et lon ?

5.3.2 geo_distance — recherche par rayon

Trouver tous les coursiers dans un rayon de 5 km autour d'un point central :

```
GET /position/_search
{
  "query": {
    "geo_distance": {
      "distance": "5km",
      "currentPosition": {
        "lat": 45.8566,
        "lon": 2.3522
      }
    }
  }
}
```

- Combien de positions sont dans ce rayon ?
- Réduire le rayon à 4km — combien reste-t-il ?
- Essayer avec une unité différente : "distance": "10mi" (miles)

Trier par distance

Retourner les positions les plus proches en premier, avec la distance calculée :

```
GET /position/_search
{
  "query": {
    "geo_distance": {
      "distance": "10km",
      "currentPosition": { "lat": 45.8566, "lon": 2.3522 }
    }
  },
  "sort": [
    {
      "_geo_distance": {
```

```
        "currentPosition": { "lat": 45.8566, "lon": 2.3522 },
        "order": "asc",
        "unit": "km"
      }
    }
  ],
  "size": 10
}
```

- Le premier résultat est-il bien le plus proche ?
- La valeur sort[0] correspond-elle à la distance en km ?

5.3.3 geo_bounding_box — recherche dans un rectangle

Trouver tous les coursiers dans une zone rectangulaire (bounding box) :

```
GET /position/_search
{
  "query": {
    "geo_bounding_box": {
      "currentPosition": {
        "top_left": {
          "lat": 46.0,
          "lon": 2.0
        },
        "bottom_right": {
          "lat": 45.7,
          "lon": 2.7
        }
      }
    }
  }
}
```

- Comparer avec geo_distance — lequel est plus rapide à exécuter ? Pourquoi ?

> *geo_bounding_box est généralement plus rapide que geo_distance car il utilise des comparaisons simples sur lat/lon sans calcul trigonométrique.*

5.3.4 geo_polygon — zone arbitraire

Définir un polygone personnalisé pour délimiter une zone géographique :

```
GET /position/_search
{
  "query": {
    "geo_polygon": {
      "currentPosition": {
        "points": [
          { "lat": 45.90, "lon": 2.25 },
          { "lat": 45.90, "lon": 2.45 },
          { "lat": 45.75, "lon": 2.45 },
          { "lat": 45.75, "lon": 2.25 },
          { "lat": 45.90, "lon": 2.25 }
        ]
      }
    }
  }
}
```

- Ce polygone correspond à quelle zone géographique approximative ?
- Ajouter un filtre supplémentaire pour ne garder que les coursiers actifs

5.3.5 Agrégations géospatiales

geo_distance — répartition par zones concentriques

Compter les coursiers par anneau de distance autour d'un point :

```
GET /position/_search
{
  "size": 0,
  "aggs": {
    "zones_distance": {
      "geo_distance": {
        "field": "currentPosition",
        "origin": { "lat": 45.8566, "lon": 2.3522 },
        "unit": "km",
        "ranges": [
          { "key": "< 2km",        "to": 2 },
          { "key": "2 - 5km",   "from": 2,  "to": 5 },
          { "key": "5 - 10km", "from": 5,  "to": 10 },
          { "key": "> 10km",    "from": 10 }
        ]
      }
    }
  }
}
```

- Quelle zone concentre le plus de coursiers ?
- La distribution est-elle uniforme ou concentrée au centre ?

geohash_grid — carte de densité

Diviser la carte en cellules geohash et compter les positions par cellule :

```
GET /position/_search
{
  "size": 0,
  "aggs": {
    "carte_densite": {
      "geohash_grid": {
        "field": "currentPosition",
        "precision": 4
      }
    }
  }
}
```

- Combien de cellules différentes contiennent des positions ?
- Augmenter la précision à 5 ou 6 — comment évolue le nombre de cellules ?

> *La précision geohash va de 1 (cellule de 5000km) à 12 (cellule de quelques cm). Une précision de 4 correspond à des cellules d'environ 40km × 20km.*

geo_centroid — centre de gravité

Calculer le centre géographique de la flotte de coursiers :

```
GET /position/_search
{
  "size": 0,
  "aggs": {
    "centre_flotte": {
      "geo_centroid": {
        "field": "currentPosition"
      }
```

```
    }
  }
}
```

- À quelle ville/zone correspond ce centroïde ?
- Calculer le centroïde pour chaque coursier séparément — ajouter une sous-agrégation terms sur l'id du coursier

5.3.6 Combiner géolocalisation et requête

Cas concret : trouver les 5 coursiers les plus proches d'une adresse de livraison, disponibles (stock > 0) :

```
GET /position/_search
{
  "query": {
    "bool": {
      "filter": [
        {
          "geo_distance": {
            "distance": "4km",
            "currentPosition": { "lat": 45.8566, "lon": 2.3522 }
          }
        }
      ]
    }
  },
  "sort": [
    {
      "_geo_distance": {
        "currentPosition": { "lat": 45.8566, "lon": 2.3522 },
        "order": "asc",
        "unit": "m"
      }
    }
  ],
  "size": 5
}
```

- Quelle est la distance du coursier le plus proche (en mètres) ?
- Modifier pour chercher dans un rayon de 500m — y a-t-il des résultats ?

Ateliers 6 — OpenSearch Dashboards

> *Objectif : créer des index patterns, explorer les données avec Discover, construire des visualisations et assembler un tableau de bord interactif sur l'index produits-java, puis une carte géospatiale sur l'index position.*

Plan de l'atelier :

1	**Index Patterns**	Créer les index patterns produits-java et position
2	**Discover**	Explorer les données, filtres DQL, champs statistiques
3	**Visualisations**	Bar chart, Pie, Data Table, Metric sur produits-java
4	**Tableau de bord**	Assembler, filtrer, drill-down, sauvegarder
5	**Carte géospatiale**	Visualisation Maps sur l'index position

---

6.1 Créer les Index Patterns

Un Index Pattern indique à Dashboards quels index OpenSearch interroger et quel champ utilise le filtre temporel.

Index Pattern produits-java

Aller dans Stack Management → Index Patterns → Create index pattern :

- Name : produits-java
- Index pattern : produits-java (pas de wildcard — index unique)
- Time field : dateAjout
- Cliquer Create index pattern

Explorer les champs détectés :

- Quels champs sont Searchable ? Aggregatable ?
- Pourquoi le champ nom est-il searchable mais pas aggregatable ?
- Le champ categorie est-il aggregatable ? Pourquoi (keyword vs text) ?

> *Un champ text analysé ne peut pas être agrégé directement. Le multi-field categorie.keyword le permet. Si votre mapping a été créé avec l'atelier 4.1, les deux sont présents.*

Index Pattern position

Créer un second index pattern pour les données géospatiales :

- Name : position
- Index pattern : position
- Time field : @timestamp
- Vérifier que le champ currentPosition est détecté comme geo_point

---

6.2 : Explorer avec Discover

> ℹ️ *Objectif : Discover est l'outil d'exploration des données brutes — histogramme temporel, liste de documents et statistiques sur les champs. Cet atelier vous familiarise avec DQL et PQL*

Première exploration

Aller dans *Discover* → sélectionner l'index pattern produits-java :

- Combien de documents sont présents au total ?
- Cliquer sur un document pour voir ses champs détaillés
- Ajouter les colonnes nom, prix, categorie via le panneau gauche (Available fields)
- Quel est l'ordre de tri par défaut ?

Recherche DQL

La DQL (Dashboards Query Language) permet des recherches textuelles et structurées :

Recherche sur un champ keyword (exacte) :

```
categorie: "Informatique"
```

- Combien de produits Informatique ?

Plage de valeurs :

```
prix > 100 and prix < 300
```

- Combien de produits entre 100 et 300 € ?

Wildcard :

```
nom: "Produit*"
```

- Tous les produits dont le nom commence par Produit

Combinaison :

```
categorie: "Audio" AND prix > 50
```

- Combien de produits Audio à plus de 50 € ?

Recherche PPL

> ℹ️ Dans Discover, basculer vers PPL via le sélecteur de langage en haut à gauche de la barre de recherche.

Recherche sur un champ keyword (exacte) :

```
source=produits-java | where categorie = "Informatique"
```

- Retrouvez-vous le même nombre de produits Informatique qu'en DQL ?

Plage de valeurs :

```
source=produits-java | where prix > 100 and prix < 300
```

- Même résultat qu'avec la requête DQL équivalente ?

Wildcard :

```
source=produits-java | where like(nom, "Produit%")
```

- Notez la différence de syntaxe : PPL utilise `like()` avec `%` (style SQL) là où DQL utilise *

Combinaison avec projection des colonnes :

```
source=produits-java | where categorie = "Audio" and prix > 50 |
fields nom, prix, stock
```

- La commande `fields` permet de choisir les colonnes directement dans la requête — sans passer par le panneau Available fields

Tri et limite :

```
source=produits-java | where categorie = "Informatique" and actif
= true | sort - prix | head 10
```

- `sort - prix` trie par prix décroissant (`-` = DESC, + ou rien = ASC)
- `head 10` limite aux 10 premiers résultats

Statistiques agrégées :

```
source=produits-java | stats count() by categorie | sort - count()
```

- Comparer avec les statistiques du panneau Available fields sur le champ `categorie` — les résultats sont-ils cohérents ?

Filtres visuels

Les filtres visuels s'ajoutent en cliquant sur les valeurs dans les documents :

- Cliquer sur une valeur de categorie dans un document → Add filter
- Le filtre apparaît sous la barre de recherche — observer le badge coloré
- Cliquer sur le filtre → Edit filter → explorer les options (is not, exists…)
- Ajouter un second filtre sur actif : true
- Les filtres se combinent en AND automatiquement
- Désactiver un filtre sans le supprimer en cliquant sur le toggle

> *Les filtres visuels sont convertibles en Query DSL — cliquer sur le filtre → Edit → changer le mode de l'éditeur. Utile pour comprendre la syntaxe DSL.*

Statistiques sur les champs

Cliquer sur un champ dans le panneau *Available fields* pour voir ses statistiques :

- ***prix*** : quelles sont les valeurs min, max, médiane ?
- ***categorie*** : quelles sont les 5 valeurs les plus fréquentes ?
- ***actif*** : quelle est la proportion true / false ?

Sauvegarder une recherche

Configurer la recherche suivante et la sauvegarder :

- Requête DQL : categorie: "Informatique" AND actif: true
- Colonnes affichées : nom, prix, stock
- Tri : prix DESC
- Cliquer Save → nommer : Produits Informatique actifs
- Cette recherche sauvegardée pourra être intégrée dans un dashboard

---

6.3 : Créer des visualisations

Aller dans ***Visualize → Create new visualization***.

Chaque visualisation sera sauvegardée séparément avant d'être intégrée dans le dashboard.

Metric — nombre total de produits

Visualize → Create new → Metric → Index pattern : produits-java

- Metrics → Aggregation : Count
- Laisser toutes les options par défaut
- Cliquer sur le bouton ▶️ (Apply changes) pour prévisualiser
- Save → nommer : Nombre total de produits

Pie chart — répartition par catégorie

Create new → Pie → Index pattern : produits-java

- Metrics → Slice size → Aggregation : Count
- Buckets → Add → Split slices → Aggregation : Terms
- Field : categorie → Order by : metric Count → Size : 6
- Cliquer ▶️ — le pie doit afficher les 6 catégories
- Options → Legend position : Right
- Save → nommer : Répartition par catégorie

Bar chart vertical — prix moyen par catégorie

Create new → Vertical Bar → Index pattern : produits-java

- Metrics → Y-axis → Aggregation : Average → Field : prix
- Buckets → X-axis → Aggregation : Terms → Field : categorie
- Cliquer ▶️ — une barre par catégorie avec le prix moyen
- Options → cocher Show values on chart
- Save → nommer : Prix moyen par catégorie

Data Table — top 10 produits les plus chers

Create new → Data Table → Index pattern : produits-java

- Metrics → Count (laisser par défaut pour commencer)
- Buckets → Split rows → Aggregation : Terms

- Field : nom → Order by : Custom metric → Aggregation : Max → Field : prix
- Size : 10 → Cliquer ▶️
- Ajouter une 2e métrique : Metrics → Add → Average → Field : prix
- Ajouter une 3e métrique : Metrics → Add → Max → Field : stock
- Save → nommer : Top 10 produits les plus chers

Histogram — distribution des prix

Create new → Vertical Bar → Index pattern : produits-java

- Metrics → Y-axis → Count
- Buckets → X-axis → Aggregation : Histogram → Field : prix → Interval : 50
- Cliquer ▶️ — un histogramme des prix par tranche de 50 €
- Save → nommer : Distribution des prix

6.4 : Assembler le tableau de bord

Créer le dashboard

Dashboard → Create new dashboard :

- Cliquer Add → sélectionner les 5 visualisations créées
- Repositionner par drag & drop : Metric en haut, Pie et Bar sur la même ligne, Table et Histogram en bas
- Redimensionner les panneaux en tirant le coin bas-droit

> ℹ️ *Astuce : maintenir Shift en redimensionnant pour garder les proportions. Les panneaux s'ancrent automatiquement sur une grille invisible.*

Filtres globaux et drill-down

Tester les interactions entre panneaux :

- Cliquer sur une tranche du Pie chart (ex: Informatique) → tous les panneaux se filtrent
- Observer la barre de filtres en haut — un filtre categorie:Informatique est apparu
- Cliquer sur ce filtre → Temporarily disable — les données reviennent sans supprimer le filtre
- Cliquer sur une barre du Bar chart → double drill-down Pie + Table

Ajouter un filtre permanent

Filtrer définitivement le dashboard sur les produits actifs :

- Add filter → Field : actif → Operator : is → Value : true
- Cocher Pin across all apps — ce filtre persistera même en naviguant ailleurs
- Observer comment les chiffres du Metric changent

Recherche sauvegardée dans le dashboard

Ajouter la recherche Discover sauvegardée à l'étape 2 :

- Add → Saved searches → Produits Informatique actifs
- Elle s'affiche comme un tableau avec les colonnes configurées dans Discover
- C'est utile pour donner accès aux données brutes directement dans le dashboard

Sauvegarder et partager

Sauvegarder le dashboard :

- Save → Titre : Catalogue Produits → Description : Dashboard de suivi du catalogue produits
- Share → Get links → Permalien → copier l'URL
- Ouvrir l'URL dans un onglet privé — le dashboard s'affiche sans avoir à naviguer
- Share → Download PDF (si les rapports sont activés)

6.5 : Carte géospatiale (index position)

> ℹ️ *Cette étape utilise l'index position alimenté par l'atelier Kafka Connect. Vérifier que l'index contient des données : GET /position/_count*

Créer une visualisation Maps

Visualize → Create new → Maps

Ajouter une couche de données :

- Add layer → Documents → Index pattern : position
- Dans Geospatial field : sélectionner position (geo_point)
- Cliquer Add layer

Configurer l'affichage

Dans le panneau de configuration de la couche :

- Layer name : Positions coursiers
- Scaling → Apply global search to layer : activé
- Tooltips → ajouter les champs du document (id coursier si présent, @timestamp)
- Symbology → Fill color → By value → Field : (choisir un champ numérique si disponible)
- Cliquer Save & close

Ajouter une couche de densité (Cluster)

Pour visualiser les zones de forte concentration :

- Add layer → Cluster → Index pattern : position
- Geospatial field : position
- Cliquer Add layer

- Ajuster la taille maximale des clusters pour mieux distinguer les zones à fortes densités
- Cliquer Save & close

Explorer la carte

- Zoomer sur la zone d'activité principale
- Utiliser le filtre spatial : Draw rectangle sur la carte → filtre automatique
- Dans la barre de recherche DQL, tester un filtre pour isoler un coursier spécifique
- Sauvegarder la carte → nommer : Positions de la flotte

Intégrer la carte dans le dashboard

Optionnel — ajouter la carte au dashboard Catalogue Produits ou créer un nouveau dashboard Suivi de la flotte :

- *Dashboard → Create new → Add → Positions de la flotte*
- Ajouter une Metric avec le count de l'index position
- Sauvegarder → Suivi de la flotte

> ***Pour aller plus loin : Explorer les visualisations TSVB (Time Series Visual Builder) pour des métriques temporelles avancées, et les alertes dans Alerting → Monitors pour déclencher des notifications sur seuils.***

Ateliers 7 — Administration et sécurité

7.1 Gestion des nœuds

Les nœuds d'un cluster OpenSearch peuvent être spécialisés pour séparer les responsabilités : gestion du cluster, stockage des données, coordination des requêtes. Cette partie couvre la spécialisation, l'ajout et le retrait à chaud, et le redémarrage propre.

7.1.1 Rôles des nœuds

L'objectif est de modifier le docker compose pour obtenir une architecture spécialisée :

Conteneur	Rôle OpenSearch	node.roles
`os-manager1`	Cluster manager dédié	`cluster_manager`
`os-data1`	Nœud data n°1	`data`
`os-data2`	Nœud data n°2	`data`
`os-coord1`	Coordinating only	`(liste vide)`

> ℹ️ *Un nœud coordinating reçoit les requêtes clients, les distribue aux nœuds data, puis agrège les résultats. Il ne stocke rien et soulage les nœuds data/manager.*

Modifier le docker-compose.yml

Adaptez chaque service pour ajouter la variable d'environnement node.roles. Voici la configuration à appliquer sur chaque service :

os-manager1 — cluster_manager dédié

```
environment:
  - node.name=os-manager1
  - node.roles=cluster_manager
  - cluster.initial_cluster_manager_nodes=os-manager1
  - discovery.seed_hosts=os-manager1,os-data1,os-data2,os-coord1
```

os-data1 et os-data2 — nœuds data

```
environment:
  - node.name=os-data1   # adapter pour os-data2
  - node.roles=data
  - discovery.seed_hosts=os-manager1,os-data1,os-data2,os-coord1
```

os-coord1 — nouveau service coordinating (à ajouter)

```
os-coord1:
  image: opensearchproject/opensearch:2.13.0
  container_name: os-coord1
  environment:
    - node.name=os-coord1
    - node.roles=[]            # liste vide = coordinating only
    - cluster.name=${CLUSTER_NAME}
    - discovery.seed_hosts=os-manager1,os-data1,os-data2,os-coord1
    - bootstrap.memory_lock=true
    - OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m
    - plugins.security.disabled=true
  ulimits:
```

```
    memlock: { soft: -1, hard: -1 }
  networks: [opensearch]
```

Démarrer le cluster spécialisé

- Arrêter l'ancien cluster puis relancer avec la nouvelle config :

```
docker compose down
docker compose up -d
```

- Attendre 30 secondes puis vérifier les rôles de chaque nœud :

```
curl -s 'http://localhost:9200/_cat/nodes?
v&h=name,node.role,heap.percent,ram.percent'
```

Colonne node.role : m = cluster_manager éligible, d = data, - = coordinating

- Identifier quel nœud a été élu cluster_manager actif :

```
curl -s 'http://localhost:9200/_cat/nodes?v&h=name,node.role,master'
```

L'astérisque * dans la colonne master indique le cluster_manager actif.

- Créer un index de test pour observer la répartition :

```
curl -s -X PUT 'http://localhost:9200/tp-shards' \
  -H 'Content-Type: application/json' \
  -d '{"settings":{"number_of_shards":4,"number_of_replicas":1}}'

curl -s 'http://localhost:9200/_cat/shards/tp-shards?v'
```

⚠️ *Si le cluster reste YELLOW après la création de l'index, c'est normal avec 2 nœuds data et 1 réplica : un shard primaire et son réplica ne peuvent pas être sur le même nœud. Avec 2 nœuds data, tous les primaires ET réplicas peuvent être alloués.*

Pour vérifier les causes d'une non-allocation :

```
curl -s 'http://localhost:9200/_cluster/allocation/explain?pretty'
```

7.1.2 — Ajout d'un nœud data à chaud

Objectif : augmenter la capacité de stockage sans interrompre le service. OpenSearch rééquilibre automatiquement les shards à l'arrivée du nouveau nœud.

- Ajouter le service os-data3 dans le docker-compose.yml :

```
os-data3:
  image: opensearchproject/opensearch:3.4.0
  container_name: os-data3
  environment:
    - node.name=os-data3
    - node.roles=data
    - cluster.name=${CLUSTER_NAME}
    - discovery.seed_hosts=os-manager1,os-data1,os-data2,os-data3,os-coord1
    - bootstrap.memory_lock=true
    - OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m
    - plugins.security.disabled=true
  ulimits:
    memlock: { soft: -1, hard: -1 }
  networks: [opensearch]
```

- Démarrer uniquement le nouveau nœud (sans toucher aux autres) :

```
docker compose up -d os-data3
```

- Observer l'arrivée du nœud et le rééquilibrage automatique des shards :

```
# Rafraîchir toutes les 2 secondes — Ctrl+C pour arrêter
watch -n 2 'curl -s http://localhost:9200/_cat/shards/tp-shards?v | head -20'
```

Observer la colonne state : STARTED = alloué, RELOCATING = en cours de migration.

- Une fois stable, vérifier la répartition finale et l'espace disque par nœud :

```
curl -s 'http://localhost:9200/_cat/allocation?v'
```

💡 *Le rééquilibrage est contrôlé par cluster.routing.rebalance.enable. Sa valeur par défaut all permet à OpenSearch de déplacer les shards automatiquement pour équilibrer la charge.*

7.1.3 Retrait d'un nœud data (décommission)

La procédure correcte est en 3 temps : exclure le nœud de l'allocation → attendre la migration complète → arrêter le conteneur.

- Exclure os-data3 de l'allocation (les shards vont migrer vers os-data1 et os-data2) :

```
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"transient":{"cluster.routing.allocation.exclude._name": "os-data3"}}'
```

- Surveiller la migration jusqu'à ce que os-data3 n'ait plus aucun shard :

```
watch -n 2 'curl -s http://localhost:9200/_cat/shards/tp-shards?v | grep os-data3'
# Quand la commande précédente ne retourne plus rien, continuer :
curl -s 'http://localhost:9200/_cluster/health?pretty'
```

- Attendre le statut GREEN puis arrêter le conteneur :

```
docker compose stop os-data3 && docker compose rm -f os-data3
```

- Annuler la règle d'exclusion (important pour la suite) :

```
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"transient":{"cluster.routing.allocation.exclude._name": null}}'
```

⚠️ *Ne jamais arrêter un nœud data sans l'avoir exclu au préalable. Si les shards primaires hébergés sur ce nœud n'ont pas eu le temps de migrer, le cluster passe en RED.*

7.1.4 Redémarrage propre du cluster

Lors d'une maintenance planifiée (mise à jour, reconfiguration), le redémarrage doit se faire en suivant une procédure précise pour éviter les réallocations inutiles et accélérer la récupération.

Cas A — Redémarrage d'un seul nœud

- Suspendre l'allocation des réplicas (évite les migrations inutiles pendant le redémarrage) :

```
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"transient":{"cluster.routing.allocation.enable": "primaries"}}'
```

- Vider le translog pour un redémarrage propre :

```
curl -s -X POST 'http://localhost:9200/_flush'
```

- Redémarrer le nœud :

```
docker compose restart os-data1
```

- Réactiver l'allocation complète dès que le nœud est de retour :

```
# Attendre que le nœud réapparaisse :
curl -s 'http://localhost:9200/_cat/nodes?v&h=name,node.role'
# Réactiver :
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"transient":{"cluster.routing.allocation.enable": null}}'
```

Cas B — Redémarrage complet du cluster

- Désactiver le rebalancing pour éviter les va-et-vient au redémarrage :

```
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"persistent":{"cluster.routing.rebalance.enable": "none"}}'
```

- Arrêter le cluster :

```
docker compose down
```

- Redémarrer dans l'ordre — le cluster_manager en premier :

```
docker compose up -d os-manager1
sleep 15
docker compose up -d os-data1 os-data2
sleep 5
```

```
docker compose up -d os-coord1
docker compose up -d opensearch-dashboards
```

- Attendre GREEN puis réactiver le rebalancing :

```
curl -s 'http://localhost:9200/_cluster/health?
wait_for_status=green&timeout=120s&pretty'
curl -s -X PUT 'http://localhost:9200/_cluster/settings' \
  -H 'Content-Type: application/json' \
  -d '{"persistent":{"cluster.routing.rebalance.enable": null}}'
```

ℹ️ *Lors du redémarrage complet, chaque nœud data retrouve ses données sur le volume Docker. OpenSearch reconstruit l'état du cluster depuis les métadonnées persistées, sans re-copier les données.*

7.2 Snapshots

Un repository snapshot définit l'emplacement de stockage des sauvegardes. Le type filesystem est le plus simple pour un lab : il utilise un répertoire partagé monté sur tous les nœuds.

7.2.1 Configurer un repository filesystem

Ajouter le volume partagé au docker-compose.yml

Modifier chaque service OpenSearch (manager + data) pour monter le répertoire de snapshots et déclarer path.repo :

```
# Dans chaque service os-manager1, os-data1, os-data2 :
environment:
  - path.repo=/mnt/snapshots
volumes:
  - ./snapshots:/mnt/snapshots
```

- Créer le répertoire local et redémarrer le cluster :

```
mkdir -p ./snapshots
docker compose down && docker compose up -d
```

- Enregistrer le repository dans OpenSearch :

```
curl -s -X PUT 'http://localhost:9200/_snapshot/local-repo' \
  -H 'Content-Type: application/json' \
  -d '{
    "type": "fs",
    "settings": { "location": "/mnt/snapshots", "compress": true }
  }'
```

- Vérifier que le repository est accessible :

```
curl -s 'http://localhost:9200/_snapshot/local-repo?pretty'
```

7.2.2 Créer et restaurer un snapshot

- Indexer quelques documents de test :

```
for i in $(seq 1 10); do
  curl -s -X POST 'http://localhost:9200/tp-snap/_doc' \
    -H 'Content-Type: application/json' \
    -d "{\"msg\":\"document $i\",\"ts\":\"$(date -u +%Y-%m-%dT%H:%M:%SZ)\"}" >
/dev/null
done
echo '10 documents indexés'
```

- Créer un snapshot :

```
curl -s -X PUT 'http://localhost:9200/_snapshot/local-repo/snap-01' \
  -H 'Content-Type: application/json' \
  -d '{
    "indices": "tp-snap,tp-shards",
    "ignore_unavailable": true,
```

```
    "include_global_state": false
  }'
```

- Vérifier l'état du snapshot (attendre state: SUCCESS) :

```
curl -s 'http://localhost:9200/_snapshot/local-repo/snap-01?pretty'
```

- Simuler une perte de données et restaurer :

```
# Supprimer l'index
curl -s -X DELETE 'http://localhost:9200/tp-snap'

# Restaurer depuis le snapshot
curl -s -X POST 'http://localhost:9200/_snapshot/local-repo/snap-01/_restore' \
  -H 'Content-Type: application/json' \
  -d '{"indices": "tp-snap", "index_settings": {"index.number_of_replicas": 0}}'

# Surveiller la restauration
curl -s 'http://localhost:9200/_cat/recovery/tp-snap?v'
```

- Vérifier que les données sont bien restaurées :

```
curl -s 'http://localhost:9200/tp-snap/_count?pretty'
# Résultat attendu : count: 10
```

💡 *Via Dashboards : Index Management ▸ Snapshot Management pour planifier des snapshots automatiques via une interface graphique.*

7.3 Activation de la sécurité TLS

OpenSearch Security repose sur deux couches TLS indépendantes : la couche transport (communications inter-nœuds, obligatoire) et la couche HTTP (API REST, fortement recommandée). Nous allons générer les certificats nécessaires puis activer le plugin.

7.3.1 : Générer les certificats

Nous utilisons OpenSSL pour créer une CA auto-signée, puis signer les certificats nœuds et admin.

- Créer le répertoire certs et générer la CA root :

```
mkdir -p certs && cd certs

# CA root
openssl genrsa -out root-ca-key.pem 2048
openssl req -new -x509 -sha256 -key root-ca-key.pem \
  -subj '/CN=root-ca' -out root-ca.pem -days 730
```

- Générer le certificat admin (utilisé par securityadmin.sh) :

```
openssl genrsa -out admin-key-temp.pem 2048
openssl pkcs8 -inform PEM -outform PEM -in admin-key-temp.pem \
  -out admin-key.pem -nocrypt
openssl req -new -key admin-key.pem -subj '/CN=admin' -out admin.csr
openssl x509 -req -in admin.csr -CA root-ca.pem -CAkey root-ca-key.pem \
  -CAcreateserial -sha256 -out admin.pem -days 730
```

- Générer le certificat nœud (partagé par tous les nœuds en lab) :

```
openssl genrsa -out node-key-temp.pem 2048
openssl pkcs8 -inform PEM -outform PEM -in node-key-temp.pem \
  -out node-key.pem -nocrypt
openssl req -new -key node-key.pem -subj '/CN=node' -out node.csr

# SANs : tous les noms de conteneurs + localhost
echo 'subjectAltName=DNS:localhost,DNS:os-manager1,DNS:os-data1,DNS:os-data2,DNS:os-
coord1' \
  > node-ext.cnf
openssl x509 -req -in node.csr -CA root-ca.pem -CAkey root-ca-key.pem \
  -CAcreateserial -sha256 -out node.pem -days 730 -extfile node-ext.cnf

# Réutiliser le même certificat pour la couche HTTP
cp node.pem http.pem && cp node-key.pem http-key.pem
cd ..
```

- Vérifier les fichiers générés :

```
chmod 644 certs/*-key.pem
```

```
ls -la certs/
# Attendu : root-ca.pem, root-ca-key.pem, admin.pem, admin-key.pem,
#           node.pem, node-key.pem, http.pem, http-key.pem
```

7.3.2 Mettre à jour docker-compose.yml

Supprimer la variable plugins.security.disabled=true et ajouter les variables TLS ainsi que le volume certs sur chaque service OpenSearch.

Variables à ajouter sur chaque service OpenSearch (manager + data + coord) :

```
environment:
  # Supprimer : - plugins.security.disabled=true
  # Définir le mot de pase initial
  - OPENSEARCH_INITIAL_ADMIN_PASSWORD=Admin@1234!

  # Transport layer (inter-nœuds)
  - plugins.security.ssl.transport.pemcert_filepath=certs/node.pem
  - plugins.security.ssl.transport.pemkey_filepath=certs/node-key.pem
  - plugins.security.ssl.transport.pemtrustedcas_filepath=certs/root-ca.pem
  - plugins.security.ssl.transport.enforce_hostname_verification=false

  # REST layer (HTTPS)
  - plugins.security.ssl.http.enabled=true
  - plugins.security.ssl.http.pemcert_filepath=certs/http.pem
  - plugins.security.ssl.http.pemkey_filepath=certs/http-key.pem
  - plugins.security.ssl.http.pemtrustedcas_filepath=certs/root-ca.pem

  # DN autorisés
  - plugins.security.authcz.admin_dn=["CN=admin"]
  - plugins.security.nodes_dn=["CN=node"]

volumes:
  - ./certs:/usr/share/opensearch/config/certs
```

Adapter opensearch-dashboards pour utiliser HTTPS :

```
dashboards:
  environment:
    - OPENSEARCH_HOSTS=["https://os-coord1:9200"]
    - OPENSEARCH_USERNAME=admin
    - OPENSEARCH_PASSWORD=Admin@1234!
    - OPENSEARCH_SSL_VERIFICATIONMODE=none
```

7.3.3 Démarrer et initialiser la sécurité

- Redémarrer le cluster avec la sécurité activée :

```
docker compose down && docker compose up -d
# Attendre que les nœuds démarrent (30-60 secondes)
docker logs os-manager1 --tail 20 -f
# Attendre le message : 'Node started'
```

- Appliquer la configuration de sécurité initiale via securityadmin.sh :

```
docker exec os-manager1 bash \
  /usr/share/opensearch/plugins/opensearch-security/tools/securityadmin.sh \
  -cd /usr/share/opensearch/config/opensearch-security/ \
  -icl -nhnv \
  -cacert /usr/share/opensearch/config/certs/root-ca.pem \
  -cert  /usr/share/opensearch/config/certs/admin.pem \
  -key   /usr/share/opensearch/config/certs/admin-key.pem
# Résultat attendu : Done with success
```

- Tester la connexion sécurisée :

```
# -k pour ignorer la vérification du certificat auto-signé
curl -s -k -u admin:Admin@1234! 'https://localhost:9200/_cluster/health?pretty'
```

```
# Vérifier que HTTP non-sécurisé est bien refusé
curl -s 'http://localhost:9200/_cluster/health'
# Attendu : Connection refused ou Empty reply
```

- Accéder à Dashboards (https://localhost:5601) et se connecter avec admin / admin :

```
# Aller dans Stack Management > Security pour explorer
# les utilisateurs et rôles pré-configurés
```

⚠️ *En production, changer immédiatement le mot de passe admin via l'API : PUT /_plugins/_security/api/internalusers/admin avec un nouveau mot de passe fort.*

ℹ️ *Les certificats auto-signés générés ici sont suffisants pour un lab. En production, utiliser une PKI interne ou un outil dédié comme opensearch-onboarding ou cfssl pour gérer le cycle de vie des certificats.*

Récapitulatif des commandes clés

Opération	Commande
Vérifier les rôles des nœuds	`GET /_cat/nodes?v&h=name,node.role,master`
Répartition des shards	`GET /_cat/shards/{index}?v`
Exclure un nœud de l'allocation	`PUT /_cluster/settings (exclude._name)`
Suspendre l'allocation réplicas	`PUT /_cluster/settings (enable: primaries)`
Créer un snapshot	`PUT /_snapshot/{repo}/{nom}`
Restaurer un snapshot	`POST /_snapshot/{repo}/{nom}/_restore`
Appliquer la config sécurité	`securityadmin.sh -cd config/opensearch-security/`
Test connexion sécurisée	`curl -k -u admin:admin https://localhost:9200/...`

Ateliers 8 — Supervision et monitoring du cluster

> *Objectif : surveiller l'état et les performances d'un cluster OpenSearch via les API REST natives, le dashboard Query Insights intégré, et optionnellement Grafana.*

8.1 API _cat et diagnostics cluster

Les API _cat d'OpenSearch retournent des données tabulaires lisibles directement depuis la Dev Tools Console. C'est le premier réflexe pour diagnostiquer un problème.

> *Toutes les commandes suivantes sont à exécuter dans Dev Tools → Console (http://localhost:5601).*

8.1.1 Santé globale du cluster

```
# Vue d'ensemble — status, nb nœuds, nb shards
GET /_cluster/health?pretty

# Détail par index
GET /_cluster/health?level=indices&pretty
```

- Le status est-il green, yellow ou red ?
- Combien de shards sont unassigned ? (doit être 0 en cluster sain)
- Quelle est la différence entre active_shards et active_primary_shards ?

8.1.2 État des nœuds

```
# Vue synthétique : heap, CPU, charge, disque
GET /_cat/nodes?v&h=name,heap.percent,cpu,load_1m,disk.avail,disk.used_percent

# Statistiques détaillées JVM + OS + indices
GET /_nodes/stats/jvm,os,indices?pretty
```

- Quel nœud a le heap le plus élevé ?
- Quel est le pourcentage de disque utilisé sur chaque nœud ?

> ⚠️ *OpenSearch bloque les nouvelles indexations quand l'espace disque tombe sous 10% (high watermark) et passe en read-only à 5% (flood_stage). Surveiller disk.used_percent < 80%.*

8.1.3 État des index et shards

```
# Top 10 index par taille décroissante
GET /_cat/indices?v&s=store.size:desc
  &h=index,health,status,docs.count,store.size,pri,rep

# Shards — détecter les unassigned
GET /_cat/shards?v&h=index,shard,prirep,state,node&s=state

# Expliquer pourquoi un shard n'est pas assigné
GET /_cluster/allocation/explain
{
  "index": "produits-java",
  "shard": 0,
  "primary": false
}
```

- Y a-t-il des index en status yellow ou red ?
- Quel est l'index le plus volumineux ?

- Tous les shards sont-ils dans l'état STARTED ?

8.1.4 Thread pools — détecter la saturation

```
# Pools actifs, en queue, et rejetés
GET /_cat/thread_pool?v&h=name,active,queue,rejected&s=rejected:desc
```

- Y a-t-il des requêtes rejetées (rejected > 0) ?
- Un rejected > 0 sur le pool search ou write indique une saturation du cluster

8.1.5 Tâches en cours

```
# Tâches actives sur le cluster
GET /_cat/tasks?v&detailed

# Requêtes de recherche en attente
GET /_tasks?actions=*search*&detailed=true
```

- Y a-t-il des tâches longues en cours (> quelques secondes) ?

8.1.6 Segments — coût mémoire Lucene

```
GET /_cat/segments?v&h=index,shard,segment,size,docs.count&s=size:desc

# Nombre total de segments par index
GET /_cat/indices?v&h=index,segments.count,store.size
```

- Un grand nombre de petits segments indique qu'un forcemerge serait bénéfique

> ⚠️ *Un forcemerge est coûteux — l'exécuter uniquement sur les index read-only (index en état warm ou cold dans une politique ISM).*

8.1.7 Générer de la charge et observer

Lancer le benchmark de l'atelier 4.1 pour observer l'évolution des métriques en temps réel :

```
# Dans un terminal — lancer le benchmark
curl 'http://localhost:8080/api/produits/massive?count=50000'

# Pendant ce temps, dans Dev Tools :
# Rafraîchir toutes les 5s
GET /_cat/nodes?v&h=name,heap.percent,cpu,indexing.index_current
GET /_cat/thread_pool?v&h=name,active,queue,rejected
```

- Le heap augmente-t-il pendant l'ingestion ?
- Le thread pool write a-t-il des tâches actives ?
- Le heap redescend-il après la fin de l'ingestion ?

8.2 Query Insights (dashboard intégré)

Query Insights est un dashboard natif d'OpenSearch Dashboards qui affiche les requêtes les plus coûteuses en temps réel — sans configuration de script ni d'index externe.

8.2.1 Accéder à Query Insights

Dans OpenSearch Dashboards (http://localhost:5601) :

- Menu hamburger → Observability → Query Insights
- Si le menu n'apparaît pas : vérifier que le plugin query-insights est installé

```
# Vérifier la présence du plugin
GET /_cat/plugins?v&h=name,component,version
# Chercher : query-insights dans la liste
```

8.2.2 Activer le monitoring des Top N queries

Depuis Dev Tools, activer le suivi des requêtes lentes par latence :

```
# Activer le top N queries par latence
PUT /_cluster/settings
{
  "persistent": {
    "search.insights.top_queries.latency.enabled": true,
    "search.insights.top_queries.latency.window_size": "1m",
    "search.insights.top_queries.latency.top_n_size": 10
  }
}

# Activer aussi le suivi CPU
PUT /_cluster/settings
{
  "persistent": {
    "search.insights.top_queries.cpu.enabled": true,
    "search.insights.top_queries.cpu.window_size": "1m",
    "search.insights.top_queries.cpu.top_n_size": 10
  }
}
```

8.2.3 Générer des requêtes variées

Envoyer des requêtes de nature différente pour alimenter le dashboard :

```
# Requête simple (rapide)
GET /produits-java/_search
{ "query": { "match_all": {} }, "size": 10 }

# Requête avec agrégation (plus lente)
GET /produits-java/_search
{
  "size": 0,
  "aggs": {
    "par_cat": {
      "terms": { "field": "categorie", "size": 100 },
      "aggs": {
        "prix_stats": { "extended_stats": { "field": "prix" } }
      }
    }
  }
}

# Requête wildcard (coûteuse)
```

```
GET /produits-java/_search
{
  "query": {
    "wildcard": { "nom": { "value": "*Java*" } }
  }
}

# Répéter plusieurs fois via JMeter ou boucle bash
for i in {1..20}; do
  curl -s -X GET http://localhost:9200/produits-java/_search \
    -H 'Content-Type: application/json' \
    -d '{"size":0,"aggs":{"c":{"terms":{"field":"categorie"}}}}' > /dev/null
done
```

8.2.4 Explorer le dashboard Query Insights

Retourner dans Dashboards → Observability → Query Insights :

- Onglet Top Queries — quelles requêtes ont la latence la plus élevée ?
- Cliquer sur une requête → voir les détails : index ciblé, type, timestamp
- Onglet Live Queries — observer les requêtes en cours d'exécution
- Basculer entre Latency, CPU, Memory pour changer le critère de tri
- Modifier le sélecteur de fenêtre temporelle (1min, 5min, 1h)

8.2.5 Configurer depuis l'interface

Dans Query Insights → Configuration :

- Modifier le nombre de Top N queries à afficher (5, 10, 100)
- Changer la taille de la fenêtre d'observation
- Activer l'export vers un index local pour historiser les données

> *L'export vers un index local permet de construire des visualisations Dashboards sur l'historique des requêtes lentes — exactement comme le Stack Monitoring d'Elastic.*

8.3 Grafana

Grafana propose un dashboard OpenSearch prêt à l'emploi importable en un clic, avec toutes les métriques de production : heap, CPU, latences, taille des index, shards.

> ⚠️ *Cette partie nécessite d'ajouter Grafana au docker-compose du TP. Si le temps est limité, cette partie peut être réalisée après la formation.*

8.3.1 Ajouter Grafana au docker-compose

Ajouter ce service dans le docker-compose.yml de la stack OpenSearch :

```
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_USERS_ALLOW_SIGN_UP=false
    volumes:
      - grafana-data:/var/lib/grafana
    networks:
      - opensearch-net

# Ajouter dans la section volumes :
  grafana-data:
```

```
docker compose up -d grafana
# Grafana disponible sur http://localhost:3000
# Login : admin / admin
```

8.3.2 Installer le plugin OpenSearch pour Grafana

Dans Grafana → Administration → Plugins → chercher OpenSearch :

- Installer le plugin grafana-opensearch-datasource
- Ou via CLI :

```
docker exec grafana grafana-cli plugins install \
  grafana-opensearch-datasource
docker compose restart grafana
```

8.3.3 Configurer la datasource OpenSearch

Grafana → Connections → Data sources → Add new data source → OpenSearch :

- URL : http://opensearch-node1:9200
- Basic Auth : désactivé (sécurité désactivée dans le TP)
- Version : 2.x+
- Cliquer Save & test → doit afficher Data source is working

8.3.4 Importer le dashboard OpenSearch

Grafana → Dashboards → Import :

- Entrer l'ID : 19504 (dashboard communautaire OpenSearch Cluster Stats)
- Cliquer Load → sélectionner la datasource OpenSearch créée
- Cliquer Import

Le dashboard affiche immédiatement :

- Heap JVM (%) par nœud — évolution temporelle
- CPU utilization — charge par nœud
- Indexing rate — documents/seconde
- Search rate — requêtes/seconde
- Taille totale des index — évolution
- Nombre de shards actifs et unassigned
- GC collection time — durée de garbage collection

8.3.5 Observer pendant un benchmark

Lancer le benchmark massif depuis l'atelier 4.1 et observer Grafana en temps réel :

```
curl -X POST 'http://localhost:8080/api/produits/massive?count=50000'
```

- Le heap monte-t-il significativement pendant l'ingestion ?
- L'indexing rate monte-t-il à plusieurs milliers de docs/s ?
- Le heap redescend-il après le GC post-ingestion ?
- Comparer avec et sans setIngestionMode(true) — l'impact sur le heap est-il visible ?

8.3.6 Comparaison des dashboards

Récapitulatif de ce que propose chaque outil :

Fonctionnalité	API _cat	Query Insights	Grafana
Santé cluster	✓ temps réel	✗	✓ graphique
Heap / CPU	✓ valeur brute	✗	✓ historique
Requêtes lentes	slow logs seulement	✓ dashboard dédié	✓ via index
Indexing rate	✓ valeur brute	✗	✓ graphique
Historique	✗ temps réel	partiel	✓ configurable
Setup requis	aucun	activation cluster	docker + datasource

> *En production, les équipes utilisent généralement les 3 niveaux conjointement : _cat pour le diagnostic immédiat en ligne de commande, Query Insights pour analyser les requêtes problématiques, et Grafana (ou Datadog/CloudWatch) pour l'historique et les alertes.*

Ressources & Documentation

- Documentation officielle OpenSearch : https://opensearch.org/docs/latest/
- GitHub OpenSearch : https://github.com/opensearch-project/OpenSearch
- Data Prepper : https://opensearch.org/docs/latest/data-prepper/
- Logstash plugin OpenSearch : https://github.com/opensearch-project/logstash-output-opensearch
- ISM (Index State Management) : https://opensearch.org/docs/latest/im-plugin/ism/index/
- PPL (Piped Processing Language) : https://opensearch.org/docs/latest/search-plugins/sql/ppl/index/
- DQL (Dashboards Query Language) : https://opensearch.org/docs/latest/dashboards/dql/